

第一章 研究背景

1.1 内存管理机制的演进

内存管理是操作系统的基石，直接决定了系统的性能与稳定性。随着硬件技术的飞速发展，内存容量已从早期的兆字节（MB）级别跨越至太字节（TB）级别，然而 Linux 内核中基础的内存管理单元——页（Page）的大小，却长期停留在 4KB。这种“大内存、小页面”的矛盾，日益成为制约高性能计算和高速存储设备发挥潜力的瓶颈。

1.1.1 页表与 TLB 的工作原理

在现代计算机体系结构中，CPU 通过虚拟地址访问内存，而内存硬件使用物理地址寻址。页表（Page Table）是维护这种虚拟地址到物理地址映射关系的核心数据结构。为了加速这一频繁的转换过程，CPU 内部集成了转换后备缓冲区（Translation Lookaside Buffer, TLB）。

TLB 是一种容量极小但速度极快的硬件缓存。当 CPU 访问一个虚拟地址时，首先查询 TLB。如果命中（TLB Hit），则直接获得物理地址；如果未命中（TLB Miss），则需要遍历多级页表，这将带来巨大的 CPU 周期开销。在 4KB 小页面的机制下，覆盖同样的内存区域需要海量的页表项（PTE）。例如，映射 1GB 内存需要 262,144 个 4KB 页面。这会导致 TLB 表项被快速填满并频繁置换，引发严重的 TLB Miss，进而大幅降低系统性能。

1.1.2 4KB 页面在大内存系统中的扩展性问题

学术界也已经注意到 4KB 页面在大内存场景下面临的扩展性瓶颈。Weisberg 和 Wiseman 在《Using 4KB Page Size for Virtual Memory is Obsolete》中指出，4KB 页面虽然有利于减少外部碎片，但随着内存容量不断增长，小页会显著降低 TLB 覆盖率，其核心结论是：较大的页可以提升 TLB coverage，从而“eliminating the need to access memory resident page tables”[1]。在此之后，一系列围绕云计算与虚拟化环境的研究进一步量化了这种开销，表明在嵌套虚拟化和大内存系统中，TLB miss 及其引发的页表遍历可能占据应用执行时间的相当大比例[2][4]。

在体系结构层面，Patil 对多核体系结构上的 TLB 与 page walk 开销进行了系统评估，指出在采用基于多级 radix 树页表的体系结构（如 x86-64）中，每一次 TLB miss 都可能触发多次内存访问来完成页表遍历，而 superpage 机制的重要目标之一，

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

就是通过增加 TLB 覆盖率来降低 miss 率和 page walk 次数[5]。Sites 在 SIGARCH 技术博客中也给出了直观的上界估计：在典型的 x86-64 实现中，“each TLB miss can take up to five memory accesses to walk a page-table tree”[6]。这意味着在随机访问大工作集的负载下，即便缓存命中，地址转换本身仍可能成为性能瓶颈。

在操作系统实现层面，4KB 基本页还会通过元数据和回收路径放大性能与空间开销。Martins 在 Oracle Linux 内核博客中给出了一个直接的估算：在 x86 系统上，每个 4KB 页都对应一个约 64 字节的 struct page，“its overhead is about 1.5% of total physical memory... 16 GB per 1 TB”[7]。Corbet 在分析 struct page 重构的文章中同样指出，内核“maintains one `page` structure for every physical page of memory”，在 4KB 页大小下，一个典型系统往往要管理“millions of those structures”[8]。这意味着，一旦物理内存扩展到数百 GB 乃至 TB 级别，仅 mem_map 本身就会消耗几十 GB 的内存，并持续占用处理器缓存。

从内核开发者的角度来看，以 4KB 为基本粒度的管理模式不仅带来空间浪费，更体现在 CPU 时间开销上。Wilcox 在 2023 年的“The state of the page in 2023”报告中明确指出，相比更大的页，“managing a system with 4KB pages is inefficient at best”，因为在这种配置下，内核必须遍历海量的 `page` 结构体来完成基本的内存管理工作[11]。这些工作包括页回收（reclaim）、回写（writeback）、跨 NUMA 结点迁移等，它们都天然依赖于对 LRU 链表和 page 元数据的大量扫描，在大内存机器上会线性甚至超线性地放大 CPU 开销。

针对现有 LRU 回收机制，Zhao 及其提出的 Multi-Generational LRU（MGLRU）工作从线上系统剖析的角度给出了更为尖锐的评价：在补丁说明和后续报道中，他将当前的 page reclaim 形容为“too expensive in terms of CPU usage and it often makes poor choices about what to evict”[10]。Corbet 在对 MGLRU 的技术分析中进一步指出，当前内核需要“scan through the pages themselves, and must use reverse-mapping to find and check the associated PTEs; that is expensive”[9]。结合前述对 struct page 元数据规模的观察，这些结果共同表明：在 TB 级内存系统中，以 4KB 页面为基本单位的 TLB 管理、页表遍历以及基于 LRU 链表的页面回收，已经演变为影响整体性能和可扩展性的关键瓶颈。

1.2 Linux 文件系统 I/O 栈概述

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

Linux 的 I/O 栈是一个高度分层的复杂系统，旨在为用户提供统一的接口，同时屏蔽底层硬件的差异。其核心组件包括虚拟文件系统（VFS）、页缓存（Page Cache）、具体文件系统实现以及块设备层。

1.2.1 Page Cache 与 VFS 层交互

VFS 定义了通用的文件操作接口（`struct file_operations`）。当用户发起 `read` 系统调用时，控制流经由 VFS 层，进入具体文件系统的 `read_iter` 回调。在此过程中，Page Cache 扮演了至关重要的角色，它利用 `address_space` 结构体和 `radix tree`（或较新的 `xarray`）来缓存文件数据，减少对低速磁盘的访问。

以下代码展示了从 VFS 层到具体文件系统（以 F2FS 为例）的读取调用路径。可以看到，内核通过 `generic_file_read_iter` 最终调用 `filemap_read`，并触发 `readahead` 预读机制：

```
// File: fs/read_write.c & fs/f2fs/file.c & mm/filemap.c

// vfs read entry
ssize_t vfs_read(struct file *file, char __user *buf, size_t count, loff_t *pos)
{
    if (file->f_op->read)
        ret = file->f_op->read(file, buf, count, pos);
    else if (file->f_op->read_iter)
        ret = new_sync_read(file, buf, count, pos);
    else
        ret = -EINVAL;
    /* ... */
    return ret;
}

// F2FS's specific implementation of file_operations
static const struct file_operations f2fs_file_operations = {
    .read_iter = f2fs_file_read_iter,
    .write_iter = f2fs_file_write_iter,
    // ...
};

// read_iter
static ssize_t f2fs_file_read_iter(struct kiocb *iocb, struct iov_iter *to)
{
    // ... direct IO
    return generic_file_read_iter(iocb, to);
}
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

//filemap.c
ssize_t generic_file_read_iter(struct kiocb *iocb, struct iov_iter *iter)
{
    // ...
    return filemap_read(iocb, iter, 0);
}
// in filemap_read->filemap_get_pages->page_cache_ra_order
static void read_pages(struct readahead_control *rac)
{
    //...
    if (aops->readahead) {
        aops->readahead(rac);
        /* Clean up the remaining folios. */
        while ((folio = readahead_folio(rac)) != NULL) {
            folio_get(folio);
            filemap_remove_folio(folio);
            folio_unlock(folio);
            folio_put(folio);
        }
    } else {
        while ((folio = readahead_folio(rac)) != NULL)
            aops->read_folio(rac->file, folio);
    }
    //...
}
// F2FS's readahead function
const struct address_space_operations f2fs_dblock_aops = {
    .readahead    = f2fs_readahead,
};

```

在 `filemap_read` 中，内核会检测页面是否在缓存中。如果不在，则启动 `readahead` 算法，通过 `page_cache_ra_unbounded` 分配页面并向底层文件系统发起读取请求。

1.2.2 传统 Buffer Head 映射机制及其局限性

在引入 `iomap` 之前，Linux 文件系统长期依赖 `buffer_head` 结构来管理磁盘块与内存页的映射。`buffer_head` 最初是老式缓冲区缓存（`buffer cache`）的一部分，用于描述一个文件系统块的状态，其中包含块号、块大小、所在块设备、对应的 `struct page` 指针以及 I/O 完成回调等字段[14][15]。从设计上看，它把“4KB 页缓存中的一个块”和“块设备上的一个扇区（或文件系统块）”一一对应起来。

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

这一抽象与早期机械硬盘的物理特性高度耦合：内核通过 `buffer_head` 向上提供一个以 512 字节为基础的块映射接口，让文件系统在 4KB 页缓存之上继续细分出更小的块粒度，用于对齐磁盘扇区并进行精细化的脏标记与 I/O 提交[15]。在典型配置下，一个 4KB 页会挂接一个环形链表，链表中的每个 `buffer_head` 代表一个文件系统块，其大小通常是 512B、1KB 或 4KB。

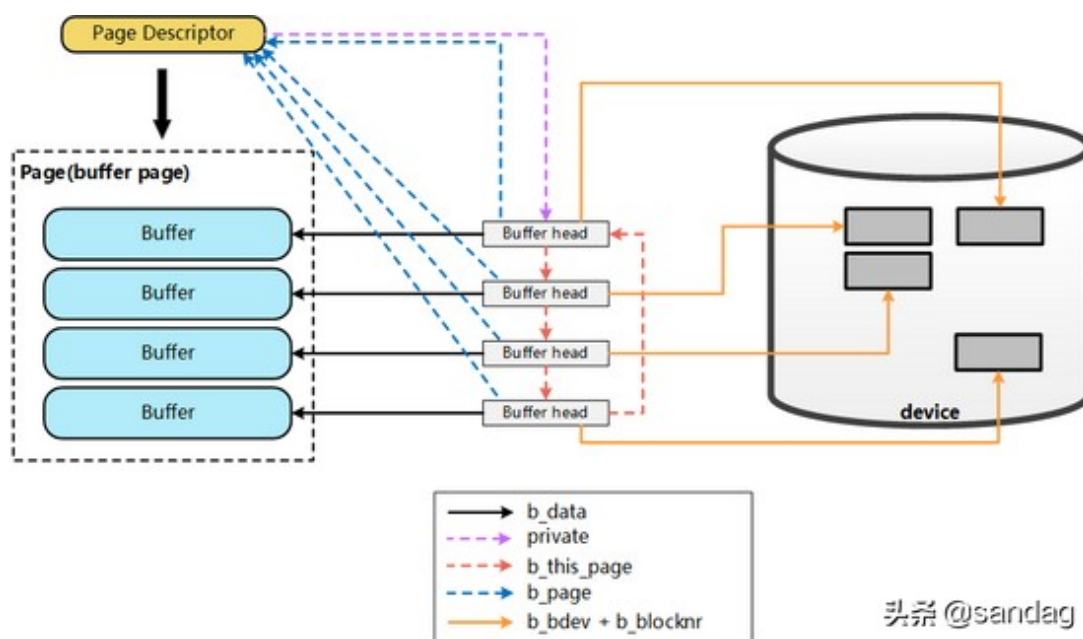


图 3-1 `buffer_head` 结构体示意图

然而，随着块层 BIO 抽象和逻辑块地址（Logical Block Address, LBA）空间的引入，底层设备已经可以直接以任意字节数目提交 I/O，请求合并和顺序调度也都在块层完成。在这种架构下，`buffer_head` 不再承担“唯一的 I/O 单元”角色，而更多地变成了一个遗留的状态容器：内核文档明确指出，在旧的 I/O 模型下，页缓存需要依赖“linked lists of buffer heads”来管理每个块的 `uptodate/dirty` 状态和写回，这在现代系统中是很低效的[16]。

具体来说，`buffer_head` 带来了三类开销。其一是元数据膨胀：每个小块都需要一个 `buffer_head`，在文件系统块大小小于页大小（例如 1KB 块配合 4KB 页缓存）时，一个页要挂接多个 `buffer_head`，这些结构体本身就占据了相当可观的内存，并持续污染 CPU cache[14][16]。其二是链表操作成本：对一个 4KB 页执行 I/O、刷新或回收时，需要遍历整条 `buffer_head` 链表，逐个更新标志位、拼接 BIO 或调用提交函数 `submit_bh`，这在高并发路径上会消耗大量 CPU 周期[14]。

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

其三则是可扩展性问题。当文件系统块大小大于页缓存粒度（即 $\text{block size} > \text{page size}$ ）时，基于 `buffer_head` 的模型几乎无法优雅支持这一用例：单个文件系统块跨越多个页，链表组织和状态同步都变得极为复杂，因此相关文档直言 “`buffer_heads` won't work very well for that”[17]。正因如此，`kernel` 文档已经将 `buffer heads` 标记为 “`deprecated`”，明确建议新文件系统应当直接使用 `iomap`，而不是继续在缓冲区缓存的遗留接口上叠加新特性[16]。

从更宏观的角度看，业界和学界在近十年都在强调“粗粒度 I/O 和内存管理”的重要性：无论是研究大页和内存连续性的 `Contiguity` 系列工作，还是面向高性能 SSD 的大块顺序 I/O 评估，都指向同一个趋势——以更大的基本单位管理数据可以显著减少元数据和地址转换成本[18]。`buffer_head` 这种基于 512B~4KB 粒度的遗留抽象，显然已经无法适应 TB 级内存和高并发 NVMe 设备的需求，这也为后续 `iomap` 框架和 `Large Folios` 的引入奠定了问题背景。

1.3 F2FS 文件系统架构

F2FS (Flash-Friendly File System) 是由三星主导设计的一种面向 NAND 闪存的日志结构文件系统，最初在 FAST'15 论文《F2FS: A New File System for Flash Storage》中系统提出[19]。与 JFFS2、UBIFS 等直接运行在裸闪存上的文件系统不同，F2FS 假定底层设备已经由 Flash Translation Layer (FTL) 进行管理，并以块设备的形式暴露给内核[20]。F2FS 的设计目标，是在不破坏这一抽象的前提下，最大化顺序写比例、降低写放大，并为移动设备和 SSD 提供稳定的长期性能[19][20]。

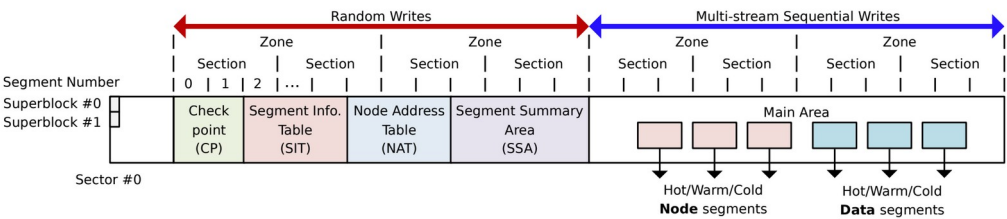


图 1-1 F2FS 磁盘布局架构图 (Superblock / Checkpoint / SIT / NAT / SSA / Main Area)

1.3.1 日志结构文件系统 (LFS) 与闪存感知设计

NAND 闪存的物理特性决定了传统“就地更新”的文件系统在闪存上会面临严重的性能和寿命问题。一方面，闪存只能按页 (page) 写入，却必须按块 (erase block) 擦除；典型配置下，一个擦除块由数百个页组成，容量从数百 KB 到数 MB 不等[19]

¹ 本章详细剖析了在 F2FS 中引入 `Large Folios` 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

[21]。在同一块中更新少量随机页面时，FTL 通常需要执行“读 - 改 - 写”操作：把整块数据读入缓存、修改其中少数页，再擦除并重写整块，这直接放大了物理写入量，即所谓写放大（Write Amplification）[19]。

另一方面，闪存块的擦写次数有限，需要通过磨损均衡（Wear-Leveling）将写入压力均匀分布到整个地址空间。如果上层文件系统频繁对同一逻辑区域进行小随机写，FTL 既难以做出良好的顺序写调度，也难以在不牺牲性能的前提下完成垃圾回收（Garbage Collection, GC）和磨损均衡[19]。长期来看，这会导致 SSD 在经历一段时间随机写负载后，顺序吞吐显著下降，表现为“写入性能随时间退化”的现象[13][19]。

为此，F2FS 采用了经典的日志结构（Log-structured）思想：所有用户数据和大部分元数据都采用顺序追加写入，禁止在原位置覆盖更新[19]。当文件发生修改时，新数据被写入主区域（Main Area）中的空闲块，旧数据所在的块被标记为无效；FTL 可以将这些顺序写映射到物理地址空间中的连续区域，从而显著降低内部读 - 改 - 写的概率，并使得底层 GC 更容易合并整块的无效页[19][21]。

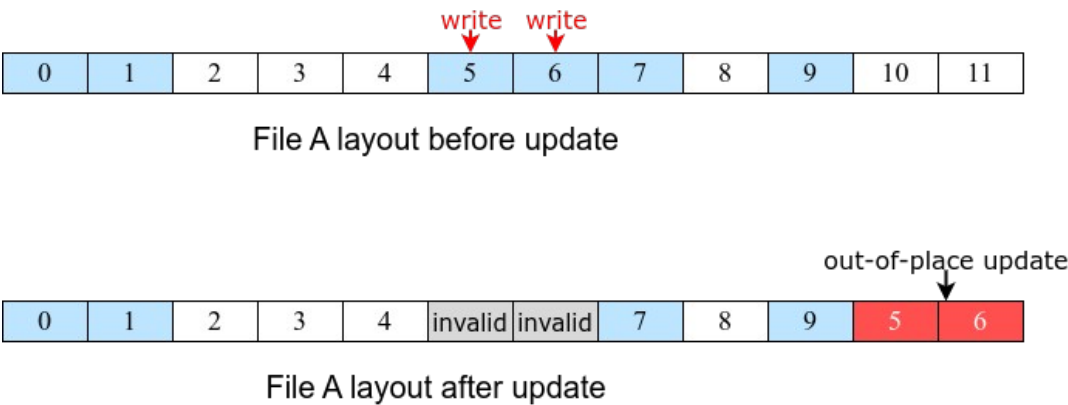


图 1-2 F2FS 日志结构写入示意图

与早期的 LFS 方案不同，F2FS 在日志结构之上加入了对“冷热数据分离”和多日志（multi-head logging）的支持：系统会根据数据的更新频率和类型，将其写入不同的日志流和物理区域，例如分别为目录元数据、热数据、冷数据分配独立的 segment 流[19]。这样一来，冷数据所在的 segment 可以长期保持较高的有效比例，避免被频繁 GC；而热数据集中在少数 segment 中，即使要频繁搬移，也不会污染大片本可保持稳定的冷数据区域。

这种从文件系统层面对闪存友好的顺序写优化，为后续在页缓存层引入 Large Folios 打下了基础。当上层能够以更大的 I/O 粒度（例如 64KB、256KB，甚至 MiB

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

级别)顺序写入时, F2FS 的日志结构可以更充分地填满一个或多个 segment, 进一步降低写放大系数, 并减少 FTL 层 GC 的元数据处理与地址转换开销[18][19]。

1.3.2 数据布局、Segment 结构与 Checkpoint 机制

在磁盘布局上, F2FS 将整个卷划分为多个逻辑区域, 包括超级块 (Superblock)、检查点区域 (Checkpoint Area)、段信息表 (Segment Information Table, SIT)、节点地址表 (Node Address Table, NAT)、段摘要区 (Segment Summary Area, SSA) 和主区域 (Main Area) [19][20]。其中, 主区域承担绝大部分用户数据和节点的存储任务, 而 SIT、NAT 和 SSA 则为日志结构写入和清理过程提供元数据支持。

F2FS 采用多级空间划分: 以 4KB 为文件系统块 (block) 的基本单位, 若干连续的块组成一个段 (segment), 多个 segment 再组成 section, 多个 section 进一步构成 zone[19][20]。在常见配置中, 一个 segment 的大小通常为数 MB 量级 (例如由 512 个 4KB 块构成约 2MB 的 segment), 而 GC 的基本工作单元往往是 section 或 zone——也就是说, F2FS 倾向于一次性搬移一大段连续的逻辑空间, 而不是在整个卷上零散地移动单个块[19][21]。这种“分级顺序写 + 大粒度清理”的布局设计, 使得 F2FS 能够更好地配合 FTL 的擦除块管理与磨损均衡策略。

在这些元数据结构中, SIT 用于记录每个 segment 的有效块数量等利用率信息, 为 GC 策略选择“肮脏度”较高的段提供依据; NAT 则维护节点 ID (NID) 到物理块地址的映射, 使得 F2FS 可以在移动节点和数据块时, 只通过修改 NAT 记录来完成地址重定向, 而无需递归修改所有父节点; SSA 存储每个 segment 的摘要信息, 便于在 GC 和恢复过程中快速定位有效数据[19][20]。

为了在崩溃后快速恢复一致状态, F2FS 设计了专门的 Checkpoint 机制。卷上预留了两个 Checkpoint 区域, 系统在挂载和定期写回时将关键的文件系统元数据——包括 SIT/NAT 的增量变化、当前日志头的位置以及脏段信息——刷入 Checkpoint 区[19]。挂载时, 内核只需选择时间戳最新且校验通过的 Checkpoint 记录, 就能恢复到最近一次一致状态; 随后再通过 SSA 所记录的 segment 摘要进行 Roll-forward, 将尚未完全持久化的最新日志片段补齐[19][20]。这一设计在保证崩溃一致性的同时, 将挂载时间控制在与设备带宽近似线性相关的范围内, 适合手机和嵌入式设备频繁重启的使用场景。

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点, 为第四章的解决方案设计提供了问题导向。

值得注意的是，F2FS 固定使用 4KB 作为文件系统块大小，这在很长一段时间里与主流内核页大小保持一致。但随着服务器和移动设备纷纷探索 16KB 甚至更大的内存页配置（例如近期 Google 在 Android 平台上引入对 16KB memory pages 的正式支持，并要求新应用完成适配[23]），文件系统与内存管理之间的粒度错配问题再次凸显。在这样的背景下，引入 Large Folios 使得内核可以在页缓存层以 16KB 乃至更大的粒度管理 I/O，而 F2FS 仍旧可以在其 4KB 块的内部布局和 segment/section/zone 级别的 GC 策略上保持不变。换言之，Large Folios 为“更大页大小”和“F2FS 的细粒度块管理”之间提供了一个自然的桥梁，同时为后续在 Android 等移动平台上推广 16KB 页大小预留了充分的演进空间[19][20][23]。

第二章 引入 Folio 和 iomap 的意义

随着存储硬件性能的指数级增长，Linux 内核传统的内存管理和文件系统 I/O 路径日益显现出疲态。为了应对“大内存”和“高性能存储”的双重挑战，内核社区引入了 Struct Folio 和 iomap 框架。本章将详细阐述这两项技术的引入背景、设计哲学及其带来的性能变革。

2.1 Struct Folio: 新一代内存管理单元

2.1.1 Linux 传统大页机制回顾：hugetlbfs、透明大页（THP）与复合页

在 Folio 出现之前，Linux 内核针对“比 4KB 更大”的内存单元，实际上走了三条路线：面向用户空间显式使用的 hugetlbfs，尝试自动折叠匿名页/页缓存的透明大页（Transparent Huge Pages, THP），以及完全在内核内部使用的复合页（Compound Page）。

hugetlbfs 通过专门的文件系统和一套显式分配接口向应用暴露预留的大页池。典型用法包括挂载 hugetlbfs、在该挂载点上 mmap 带有 MAP_HUGETLB 标志的文件，或通过 shmget(SHM_HUGETLB) 申请共享内存。运维侧需要事先通过 /proc/sys/vm/nr_hugepages 等接口预留物理连续的大页，应用侧则必须针对性地修改代码，才能享受到 TLB 命中率提升带来的收益。

然而，随着内核代码规模和硬件场景的膨胀，hugetlbfs 的维护成本和语义复杂度开始被内核社区反复质疑。Matthew Wilcox 在 2023 年的讨论中明确表示，hugetlbfs “does need to go away, but that's a big job”[11][12]，并指出造成这一局面的根源在于：hugetlbfs 历史上不断叠加了 reservation、页表共享（page table sharing）等只有它才具备的特性，这些语义既和通用内存管理代码耦合紧密，又难以复用到其他子系统。在

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

2024 年关于 hugetlbfs 统一工作的讨论中，他进一步将问题聚焦到页表遍历路径上，直言 “the biggest problem is the hugetlbfs page-table walker, which has a lot of special cases and needs to be gotten rid of, somehow”[12]。这说明 hugetlbfs 在实现层面已经演变成一个难以重构的特殊分支。

相较之下，透明大页（THP）试图在不修改应用的前提下自动利用大页。内核通过后台折叠线程和页错误处理逻辑，将一片相邻的 4KB 匿名页或页缓存页折叠为一个 2MB 的 PMD 级映射，同时提供 `madvise(MADV_HUGEPAGE / MADV_NOHUGEPAGE)` 等 hint 接口，允许应用按 VMA 粒度表达大页偏好。在理想情况下，THP 可以让热点工作集在不改变用户代码的前提下获得显著的 TLB 收益。

从实践反馈来看，固定 2MB 的 THP 同样暴露出一系列问题。首先，它在许多负载下会产生明显的内部碎片：应用的实际访问模式往往并未充分利用整整 2MB，但只要有少量 4KB 子页被写入，整个大页就会常驻内存。其次，为了维持这些大页，内核需要频繁执行内存压缩（compaction）来寻找连续的 2MB 物理空间，在长期运行的系统上，这种高阶物理块分配本身就越来越困难。一旦物理内存被碎片化，THP 分配失败率会显著升高，系统在大页折叠与回退到 4KB 小页之间反复抖动，引入额外的 CPU 开销和延迟抖动[13]。

在围绕 THP 内部碎片的讨论中，有内核开发者直接指出，问题的本质在于 internal fragmentation，即 “some of the 4KB pages within a THP were never written into”。换言之，在很多现实负载下，应用实际访问的工作集远小于 2MB，但 THP 却必须为整块 2MB 大页分配物理内存并维护页表映射，从而在空间占用和缓存局部性上都造成了浪费[10][13]。

与 hugetlbfs 和 THP 面向用户空间的大页不同，复合页（Compound Page）是内核内部用来“拼接”多个连续 4KB 页的通用机制。它通过让一个 head page 持有整体元数据，并将其后的若干 tail page 标记为附属，向上暴露出一个比 4KB 更大的逻辑页。早期的 THP 实现、文件系统页缓存的大页化、高阶伙伴分配等，都不同程度地依赖于这一机制。从抽象上看，复合页为“动态大小的大页”提供了基础设施。

不过，复合页的设计在接口层面埋下了隐患：几乎所有内存管理 API 的参数类型依然是 `struct page *`，调用者拿到的指针既可能指向 head page，也可能指向某个 tail page。如果在 tail page 上错误地加锁、调整引用计数或挂接 LRU，就可能破坏整个复合页的内部结构，形成极难排查的内存破坏 Bug。因此，大量内核代码不得不在关键

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

路径上反复调用 `compound_head()`、`PageHead()`、`PageTail()` 等辅助函数，仅仅是为了弄清楚当前手里的 `page` 究竟是什么角色，这不仅增加了代码复杂度，也让审查和维护变得异常困难。

综合来看，`hugetlbfs` 属于“强语义、强隔离但接口臃肿”的显式大页方案，THP 则倾向于“对应用透明但受制于固定 2MB 粒度和碎片化问题”的自动大页方案，而复合页虽然在内核内部提供了一个更通用的大页拼接工具，却因为 `head/tail` 语义混杂而显著增加了接口复杂度和 Bug 风险。Folio 正是在这样的背景下被提出：它试图在类型层面重新刻画“一个逻辑页”的概念，为后续的 `large folios` 和多尺寸 THP 提供一个更加干净的抽象基础。

2.1.2 Folio 在内核中的演进现状与趋势

为了解决上述 `hugetlbfs`/THP/复合页各自为政以及接口语义混乱的问题，Matthew Wilcox 自 5.15 版本开始在内核中引入了 `struct folio`。Folio 在类型系统上强制区分了“单页/复合页整体”与“页内的子页”。一个 Folio 结构体自身即代表一个完整的、物理连续的内存单元，不再需要区分头尾。

```
// File: include/linux/mm_types.h

struct folio {
    /* public: */
    unsigned long flags;
    struct list_head lru;
    struct address_space *mapping;
    pgoff_t index;
    void *private;
    atomic_t _mapcount;
    atomic_t _refcount;
#ifdef CONFIG_64BIT
    unsigned int _folio_nr_pages; // 包含的 page 数量 (2^order)
#endif
    /* ... */
};

// File: include/linux/mm.h & include/linux/pagemap.h

/* page 与 folio 的互相转换 */
#define page_folio(p) (_Generic((p), \
    const struct page *: (const struct folio *)_compound_head(p), \
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```
struct page *: (struct folio *)_compound_head(p)))

#define nth_page(page, n) ((page) + (n))
#define folio_page(folio, n) nth_page(&(folio)->page, n)

/* 典型 Folio 相关 API (节选) */
void folio_lock(struct folio *folio);
void folio_unlock(struct folio *folio);
void folio_mark_dirty(struct folio *folio);
struct page *folio_file_page(struct folio *folio, pgoff_t index);

/* 文件系统在 inode 构造时声明支持 large folios */
void mapping_set_large_folios(struct address_space *mapping);
```

引入 Folio 的核心收益在于：

1. 降低 TLB Miss：通过使用大页（如 64KB, 2MB），大幅减少了页表项数量，提高了 TLB 命中率。

2. 优化 LRU 管理：LRU 链表不再挂载数以万计的 4KB 小页，而是挂载少量的 Large Folio。这极大地减少了内存回收过程中自旋锁的竞争，提升了系统在高负载下的响应速度。

从时间线上看，Folio 技术大致经历了三个阶段：第一阶段是以 5.16 合入内核主线为起点，完成基础类型 struct folio 及其在页缓存中的落地[8][12]；第二阶段是在 5.18~6.0 期间引入 mapping_set_large_folios 等 API，使得文件系统可以选择性地启用 large folios 优化[16][18]；第三阶段则是 6.x 版本中围绕大块 I/O 和块大小大于页大小的系列工作，通过 mapping_set_folio_min_order() / mapping_set_folio_orders() 等接口进一步细化页缓存中可用的 folio 尺寸范围[16][18]。

表 2-1 Folio 在 Linux 内核中的演进时间线

内核版本	发布时间（约）	关键改动	涉及子系统	说明
5.16	2022 年初	引入 struct folio 及基础 Memory Folios 框架[1][2]	内存管理 / 页缓存	为替换 struct page 奠定基础，尚以 order-0 为主
5.18	2022 年中	提供 mapping_set_large_folios() 接口，文件系统可声明	页缓存 / 文件系统	page cache 可以按更大粒度分配缓存页

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

		支持 Large Folios[3][4]		
5.19–6.5	2022–2023	大量子系统改造为使用 folio，page cache 文档明确以 folio 为基本管理单元 [5]	页缓存 / KVM / 文件系统等	减少 compound_head() 调用和 head/tail 语义混乱
6.6 及以后	2023–2024	围绕 Large Block Size 与最小 folio order 的 API 演进（mapping_set_folio_min_order / orders 等）[3][4]	页缓存 / 文件系统	为块大小大于页面大小的系统提供支持，通过改进大块顺序 I/O 提升性能。
6.10 及以后	2024 年及以后	更进一步优化支持 16KB、64KB 等大页，内核支持扩展到更大规模的 memory folio 优化[6]	页缓存 / 文件系统	为高性能 SSD 和服务端级大内存环境提供优化，进一步减少 I/O 延迟与写放大

2.2 iomap 框架：现代化的 I/O 路径

iomap 是 Linux 内核主推的新一代文件系统 I/O 框架，旨在取代陈旧的 buffer_head 机制。内核文档指出，iomap 在缓冲 I/O 路径上 “implements nearly all the folio and pagecache management that filesystems have to implement themselves under the legacy I/O model”，而旧模型则 “managed very inefficiently with linked lists of buffer heads instead of the per-folio bitmaps that iomap uses”[16]。换言之，引入 iomap 的直接动机之一，就是把 per-block 的 buffer_head 链表重构为 per-folio 的轻量状态结构，从而更好地支持 large folios。

2.2.1 iomap 数据结构与接口定义

iomap 的核心思想是将文件系统的逻辑块映射结果抽象为一个通用的描述符。无论底层文件系统如何组织数据（Ext4 的 extent 树、XFS 的 B+ 树，或 F2FS 的 NAT 表），向上层返回的都是统一的 struct iomap 结构。在此基础上，iomap 又通过迭代器和一组回调接口，将缓冲 I/O、写回和直接 I/O 均统一在同一套抽象之下[16]。

```
// File: include/linux/iomap.h

struct iomap {
    u64 addr;      // 磁盘物理字节偏移量
    loff_t offset; // 文件逻辑字节偏移量
    u64 length;    // 映射长度 (字节)
    u16 type;      // 类型: IOMAP_MAPPED, IOMAP_HOLE, ...
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

u16 flags;
struct block_device *bdev;
struct dax_device *dax_dev;
void *inline_data;
void *private; // 文件系统私有数据 (关键扩展点)
const struct iomap_folio_ops *folio_ops;
};

```

struct iomap 的每一个实例描述了文件逻辑区间到物理存储区间的一段连续映射：offset/length 表示文件中的字节范围，addr 和 bdev/dax_dev 指定了底层设备与物理偏移，type 用于区分普通映射、空洞（HOLE）、延迟分配（DELALLOC）、未初始化区间等不同语义，private 和 folio_ops 则为文件系统预留了扩展点，可以挂接自定义的 per-folio 状态或校验逻辑[16]。

这些映射并不是孤立使用的，而是通过 struct iomap_iter 串联起来。iomap_iter 中包含当前 inode、待处理的文件偏移 pos、长度 len、已经处理的字节数 processed，以及一个嵌入式的 struct iomap，用于在循环过程中复用映射结果。通用的迭代入口 iomap_iter() 会根据文件系统提供的 struct iomap_ops::iomap_begin 回调，不断产出覆盖后续文件区间的 iomap 片段，直到整个 I/O 请求范围被消费完毕[16]。

文件系统通过 struct iomap_ops 向 iomap 注册最核心的两个回调：->iomap_begin 负责在给定文件偏移和长度下填充 struct iomap，->iomap_end 则在 I/O 结束时执行收尾工作（例如更新配额、统计信息或 unwritten extent 转换）[16]。在此之上，iomap 针对不同的 I/O 场景又定义了多组辅助接口，例如 iomap_readahead / iomap_read_folio 负责缓冲读，iomap_file_buffered_write 负责缓冲写入和延迟分配，iomap_writepages 加载 struct iomap_writeback_ops 完成脏页写回，而 iomap_dio_rw 则统一了 O_DIRECT 的读写路径[16]。

在页缓存这一层面，iomap 还提供了 struct iomap_folio_ops 来定制 per-folio 行为。内核文档给出的定义如下：

```

// File: include/linux/iomap.h / fs/iomap/buffered-io.c

struct iomap_folio_ops {
    struct folio *(*get_folio)(struct iomap_iter *iter,
                               loff_t pos, unsigned len);
    void (*put_folio)(struct inode *inode, loff_t pos,
                     unsigned copied, struct folio *folio);
};

```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```
bool (*iomap_valid)(struct inode *inode,
                   const struct iomap *iomap);
};
```

其中 `get_folio/put_folio` 允许文件系统在写入路径上为每个 `folio` 设置和提交自定义的状态，而 `iomap_valid` 则用于应对空间映射在并发 I/O 过程中发生变化的情况：文件系统可以在 `->iomap_begin` 中记录一个“有效性 cookie”，在 `iomap_valid` 中重新检查当前映射是否仍然成立，如果失效则重新取样映射[16]。通过这一机制，`iomap` 在不强制文件系统持有重锁的前提下，仍然能够在复杂并发场景下保持映射与页缓存状态的一致性。

相较于 `buffer_head`，每个 `folio` 内部的 `per-fsblock` 状态由 `iomap` 内部的轻量结构维护，对于块大小小于 `folio` 大小（ $bs < ps$ ）的情况，`iomap` 只需要为每个文件系统块维护两个状态位（`uptodate` 与 `dirty`），而整个 `folio` 只需要一个小的状态对象即可。内核文档指出，这种方案的 `per-folio` 状态结构 “is much lighter weight than struct `buffer_head` because there is only one per folio, and the per-fsblock overhead is two bits vs. 104 bytes”[16]。这也解释了为什么 `iomap` 天然更适合与 `large folios` 配合：随着 `folio` 包含的 4KB 子页数量增多，`buffer_head` 的 `per-block` 链表会线性膨胀，而 `per-fsblock` 两位状态的成本则几乎可以忽略。

在更高层次上，`iomap` 通过统一 `buffered I/O` 和 `direct I/O` 的映射逻辑，为大块顺序 I/O 和多种存储介质（包括 NVMe SSD、直连持久内存等）提供了统一的抽象。对于像 F2FS 这样希望利用 `large folios` 和大粒度 I/O 的日志结构文件系统来说，`iomap` 不仅消除了对 `buffer_head` 的依赖，还提供了丰富的钩子（如 `iomap_writeback_ops` 和 `iomap_dio_ops`），便于在大页写回和复杂的空间管理策略之间进行解耦和复用[16][17]。

2.2.2 迭代器模式与区间映射优势

传统 `get_block` 接口每次只能映射一个文件系统块（通常 4KB），处理大文件时需要循环调用数百万次。而 `iomap` 引入了 `iomap_iter` 迭代器：

```
// File: fs/iomap/buffered-io.c

// iomap 迭代器模式
while (iomap_iter(&iter, ops) > 0) {
    // iter.iomap 中包含了当前处理的大段连续区间
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。


```
// 能够一次性处理数 MB 甚至 GB 的数据
iter.processed = iomap_copy_actor(...);
}
```

这种区间化的处理方式天然契合 Large Folios。当文件系统返回一个覆盖 2MB 范围的 iomap 时，内核可以一次性分配并处理一个 2MB 的 Huge Folio，或者批量处理多个 Large Folios，从而将 I/O 路径上的 CPU 开销降低数个数量级。内核上游立志要让 iomap 变成所有文件系统事实上的统一 IO 接口，目前正往各个主流文件系统大力推广。XFS 率先完成全 IO 路径的 iomap 支持，ext4 文件系统也在逐步推进，下表展示目前内核主流文件系统的 iomap 支持情况：

表 2-2 iomap 框架在主流 Linux 文件系统的支持现状

文件系统	采用 iomap 的场景	典型内核版本	Large Folio / 大块 I/O 支持	备注
XFS	数据路径全面使用 iomap：缓冲 I/O、直接 I/O、DAX、fiemap、SEEK_DATA/SEEK_HOLE 等	4.x 即开始使用，5.x 以后成为 iomap 主要示例，6.x 中作为参考实现长期维护	完整支持 Large Folios，页缓存层基本只操作 folio 而不直接处理 struct page	iomap 的最早用户之一，也是当前特性最完整、经验最成熟的文件系统
Btrfs	目前主要在 direct I/O 路径使用 iomap，缓冲 I/O 仍保留 Btrfs 自己的复杂逻辑	Linux 5.8 起在 direct I/O 路径接入 iomap	direct I/O 绕过页缓存，对 Large Folio 的依赖相对较弱；后续计划逐步改造缓冲 I/O	内核文档与开发者 Q&A 中明确：当前仅少量 IOMAP 调用，主要用来处理 direct I/O
Ext4	direct I/O 已切换到 iomap；regular file 的缓冲 I/O 正在分阶段从 buffer_head 迁移到 iomap	direct I/O：5.5 起；缓冲 I/O：6.9 左右开始有较大规模补丁，部分发行版在评估启用	基于 iomap 的缓冲 I/O 路径设计为支持 Large Folios，大页写回依赖 per-fsblock 脏位图	当前 ext4 同时保留 buffer_head 路径，在启用某些特性（如 data=journal 等）时会回退
Ext2	direct I/O 使用 iomap；缓冲 I/O 转换工作仍在推进中	Linux 6.6 起增加 direct I/O 的 iomap 支持	Large Folio 相关行为主要由 iomap 核心负责，ext2 作为简单文件系统用于演练迁移路径	在 LSFMM 会议报告中经常被用作示例，用于说明从 get_block 接口迁移到 iomap 的步骤
F2FS	direct I/O 路径已经通过 iomap_dio_rw 等接口集成 iomap；缓冲读写 + Large Folio 支持有专门 RFC 补丁集	direct I/O 支持早于 large folio；2024 年起陆续提交“基于 F2FS 扩展 iomap 的 large folio 缓冲 I/O”补丁	目标是在缓冲读写路径上利用 Large Folios 和 per-fsblock 脏位图，降低写放大并优化顺序 I/O	当前上游仍在 review / 演进中，你的课题工作正是顺着这条路线解决 F2FS 上的工程落地问题
GFS2	在集群文件系统	5.x 期间引入，对	Large Folio 支持	邮件列表讨论中

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

	的 direct I/O 路径使用 <code>iomap_dio_rw</code> 等接口，需要与 <code>glock</code> 锁机制配合	<code>mmap + page fault</code> 与 <code>glock</code> 的交互有专门修复补丁	更多依赖 <code>iomap</code> 核心能力，本身主要用于验证集群场景下的并发与恢复语义	多次提到：需要小心处理禁用 <code>page fault</code> 与 <code>iomap_dio_rw</code> 的配合，以避免死锁
Zonefs	作为“分区即文件”的简单文件系统，数据路径几乎完全依赖 <code>iomap</code> 提供的大块顺序 I/O 抽象	随 ZNS SSD 支持在 5.x 内核中加入，并持续与 <code>iomap</code> 一起演进	天然适合大块、顺序的 I/O 模式，与 Large Folio 和大块写入高度契合	常被内核文档用作“简单文件系统”示例，用来说明在无复杂特性的前提下如何完整迁移到 <code>iomap</code>

第三章 F2FS 支持 Large Folios 的核心难点

尽管 Folio 和 `iomap` 提供了理论上的性能优势，但将其应用到 F2FS 这样一个高度复杂、具有日志结构特性且支持压缩的文件系统上，面临着巨大的工程挑战。本章将深入剖析这些核心难点。

3.1 块映射与 I/O 提交的粒度失配

3.1.1 逐页处理带来的 CPU 开销

F2FS 的原生代码深受“一页即一块”（4KB）假设的影响。在 `buffered read` 路径中，`f2fs_read_single_page` 函数每次只能处理一个 4KB 页面。即使底层存储是连续的，F2FS 也必须为 Large Folio 中的每个 4KB 子页单独查找 `dnode`、获取锁并建立映射。

这种逐页处理方式在 Large Folio 场景下会导致 CPU 流水线频繁中断，锁竞争加剧，完全抵消了大页带来的 TLB 收益。

3.1.2 Large Folio 子区间多 BIO 提交与 I/O 生命周期失配

在传统 F2FS `buffered read` 路径中，系统默认假设“一页即一块”：每个 4KB 页只对应一个 `bio`，I/O 完成回调直接以“页”为单位解锁并标记 `uptodate` 即可。这套逻辑在 `single-page buffered read` 场景下运作良好，因为“`bio` 生命周期”和“`page` 生命周期”基本是一一对应的。然而在 Large Folio 场景中，一个高阶 folio 可能需要被拆分为多个 `bio` 子区间才能完成读入，如果仍然沿用“每个 `bio` 完成就直接 `folio_end_read`”的思路，就会出现 I/O 生命周期与 folio 锁语义严重失配的问题。

具体来说，当一个 16KB 的 folio 被拆成两个 8KB 的 `bio` 提交时，如果第一个 `bio` 先完成，但第二个 `bio` 仍在队列中排队或者访问磁盘，此时就存在两种极端风险：

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

要么提前解锁整个 folio，导致上层读到一半为旧数据、一半为新数据的混合状态；要么为了规避这种风险而粗暴地把整个 folio 视为“未完成”，从而丢失子区间级别的 uptodate 信息，对后续 readahead、缓存复用和错误恢复都会产生连锁影响。

iomap 在设计上给出了一个非常优雅的解决方案：通过“双层循环 + read_bytes_pending 计数”的模式，在多子区间 bio 并发的情况下，仍然能够保证 folio 的 I/O 生命周期与其锁定/解锁语义精确对齐。外层循环负责按文件逻辑区间迭代 iomap 映射，内层循环则围绕当前映射区间依次获取多个 folio 并调用 iomap_readpage_iter 读入数据：

```
// File: fs/iomap/buffered-io.c

/* iomap_readahead & iomap_readahead_iter */
/* 外循环：按映射区间迭代 */
while (iomap_iter(&iter, ops) > 0)
    /* 内循环：处理该区间内的所有 folio */
    iter.status = iomap_readahead_iter(&iter, &ctx);

/* --- 内循环实现 --- */
static int iomap_readahead_iter(struct iomap_iter *iter,
                               struct iomap_readpage_ctx *ctx)
{
    while (iomap_length(iter)) {
        /* ... */
        ctx->cur_folio = readahead_folio(ctx->rac);
        ret = iomap_readpage_iter(iter, ctx);
        /* ... */
    }
}
```

在 iomap_readpage_iter 内部，引入了一个 per-folio 的并发控制字段 read_bytes_pending。它代表“当前 folio 上仍有多少字节处于 I/O 进行中”，每一次把 folio 的一个子区间添加到 bio 之前，都会先在 iomap_folio_state 上增加对应的字节数；而在 I/O 完成回调中，则以相同的子区间粒度递减该计数，只有当 read_bytes_pending 下降到 0 时，才真正调用 folio_end_read 结束整个 folio 的读流程。这一点可以从以下关键代码中看得很清楚：

```
// File: fs/iomap/buffered-io.c
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

/* iomap_readpage_iter 中提交前的计数增加 */
ctx->cur_folio_in_bio = true;
if (ifs) {
    spin_lock_irq(&ifs->state_lock);
    ifs->read_bytes_pending += plen; /* 关键：先增加待读字节数 */
    spin_unlock_irq(&ifs->state_lock);
}

/* 然后才添加到 bio，并且可以一次性添加 folio 的大段子区间 */
bio_add_folio_nofail(ctx->bio, folio, plen, poff);

/* I/O 完成路径 */
static void iomap_read_end_io(struct bio *bio)
{
    int error = blk_status_to_errno(bio->bi_status);
    struct folio_iter fi;

    bio_for_each_folio_all(fi, bio)
        iomap_finish_folio_read(fi.folio, fi.offset, fi.length, error);
    bio_put(bio);
}

static void iomap_finish_folio_read(struct folio *folio, size_t off,
                                    size_t len, int error)
{
    struct iomap_folio_state *ifs = folio->private;
    bool uptodate = !error;
    bool finished = true;

    if (ifs) {
        unsigned long flags;

        spin_lock_irqsave(&ifs->state_lock, flags);
        if (!error)
            uptodate = ifs_set_range_uptodate(folio, ifs, off, len);

        ifs->read_bytes_pending -= len; /* 关键：减少待读字节数 */
        finished = !ifs->read_bytes_pending; /* 是否全部完成？ */
        spin_unlock_irqrestore(&ifs->state_lock, flags);
    }
}

```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```
if (finished)
    folio_end_read(folio, uptodate); /* 只有全部完成才解锁 */
}
```

Read_bytes_pending 示意图

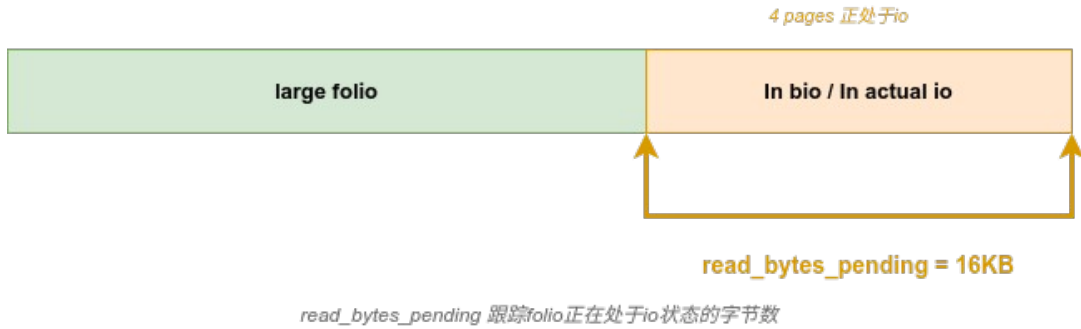


图 3-2 I/O 生命周期图

上述逻辑的关键在于“先记账，再提交”的顺序约束：只有在 `read_bytes_pending` 已经累加了本次子区间的字节数之后，才会真正调用 `bio_add_folio_nofail` 将该子区间挂到 `bio` 上。即使块层极端地做到“提交即完成”，上层在 I/O 完成回调中也一定能看到完整的待读计数，而不会出现“`bio` 已经完成、但计数尚未增加”的中间态。

为了更直观地说明这套机制为何能彻底消除 I/O 生命周期失配问题，可以用一个 16KB folio 被拆分为两个 8KB bio 的场景来刻画整个时间线：（1）T1：第一个 bio 创建，`read_bytes_pending` 增加到 8192；（2）T2：第一个 bio 提交到块设备；（3）T3：第二个 bio 创建，`read_bytes_pending` 增加到 16384；（4）T4：第二个 bio 提交到块设备；（5）T5：第一个 bio 完成，`read_bytes_pending` 从 16384 降到 8192（仍大于 0，不会解锁 folio）；（6）T6：第二个 bio 完成，`read_bytes_pending` 变为 0，此时才真正调用 `folio_end_read`。借助这套设计，无论两个 bio 的完成顺序如何，folio 都只会在“与之相关的所有 I/O 子区间全部完成”之后才被解锁，从根本上解决了 Large Folio 子区间多 BIO 并发场景下的生命周期失配问题。

3.1.3 空洞文件与 I/O 区间划分的一致性

Large Folio 带来的不仅是 I/O 并发上的生命周期问题，还暴露出 F2FS 在空洞文件（sparse file）处理上的一系列隐性假设。在原生实现中，`f2fs_map_blocks` 会通过一套较为复杂的分支来区分“dnode 是否已经分配”、“块是否为空洞”以及当前调用者希望以何种模式获取块映射（例如 PRECACHE / IOMAP 等）。在 4KB 页粒度

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

下，这些逻辑往往以“遇到空洞就 break 出循环”或“只前进一个 pgofs”来表达控制流，但在引入 Large Folios 之后，相同的语义必须被重新解释为“对一个更大范围的 I/O 区间如何切分与终止”。

例如，在 dnode 已经分配但对应逻辑块为空洞的情况下，原代码会在 PRECACHE 模式下直接跳出循环、而在 IOMAP 模式下则更新 m_next_pgofs，以便调用方在下一次迭代时从空洞之后的第一个逻辑页继续映射：

```
// File: fs/f2fs/data.c

/* f2fs_map_blocks 中处理空洞的关键分支（节选） */
} else if (is_hole) {      /* 不创建新块，但是是空洞 */
    switch (flag) {
    case F2FS_GET_BLOCK_PRECACHE:
        /* 在碰到文件空洞的情况下直接跳出循环 */
        goto sync_out;
    case F2FS_GET_BLOCK_IOMAP:
        if (map->m_next_pgofs)
            /* 记录下一个可映射的逻辑页号，供后续迭代使用 */
            *map->m_next_pgofs = pgofs + 1;
        break;
    }
}
map->m_pblk = blkaddr;
```

在 4KB 页缓存模型下，这种处理方式的含义相对直观：一次调用最多映射一个 page 对应的块，如果中途遇到空洞，就立即结束本次映射，交由上层在下一轮单页映射中继续处理。但在 Large Folio + iomap 的区间化世界里，一个 folio 通常对应连续的多个逻辑页，而 iomap_iter 又倾向于在一次迭代中返回尽可能长的“有效区间”。如果不对 f2fs_map_blocks 的空洞处理语义进行系统梳理，就容易出现以下几类问题：（1）对空洞区间的长度估计不足，导致 iomap 频繁被打断，Large Folio 无法形成大块顺序 I/O；（2）对 m_next_pgofs 更新不当，使得后续迭代直接跳过部分仍然可以按 folio 粒度读取的数据；（3）在稀疏文件的长空洞场景下，I/O 区间划分与 folio 边界错位，引发不必要的零填充和页缓存分配。

因此，在本项目的设计中，我们将“空洞文件的映射语义”也纳入了 Large Folios 适配的范围，要求所有涉及 f2fs_map_blocks 的调用路径在迁移到 iomap 之后，都能够区间级别精确地区分“真实映射区间”和“纯空洞区间”，并保证二者在 folio

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

粒度上的边界清晰可见。只有这样，才能在保持 F2FS 稀疏文件语义不变的前提下，让 Large Folios 真正发挥出顺序 I/O 和 I/O 合并方面的优势。

3.2 关键数据结构的冲突与兼容

3.2 f2fs_iomap_folio_state: folio->private 冲突与致命问题

3.2.1 功能背景

在 Linux 内核的传统设计中，`page->private`（现在是 `folio->private`）字段是一个 `void*` 类型的指针，被文件系统广泛用于存储与特定内存页相关的私有状态。F2FS 也不例外，它利用这个字段的各个比特位来存储一系列标志，如 `PAGE_PRIVATE_ATOMIC_WRITE`（原子写）、`PAGE_PRIVATE_ONGOING_MIGRATION`（GC 迁移中）等。

然而，当 `iomap` 框架引入后，它也需要使用 `folio->private` 来存储一个指向 `iomap_folio_state` 结构体的指针。这个结构体对于实现高阶 Folio 的逐块脏状态跟踪和并发 I/O 控制至关重要。也就是说，原本只被 F2FS 自己使用的 `private` 字段，现在同时被 F2FS 和 `iomap` 这两个子系统盯上了。

核心冲突 (The Core Conflict) 可以概括为：

- F2FS 的需求：需要在 `folio->private` 中存储多个布尔型标志位；
- `iomap` 的需求：需要在 `folio->private` 中存储一个指向 `iomap_folio_state` 结构体的指针。

这两者在 `folio->private` 这个有限的资源上产生了直接冲突。更复杂的是，我们为 F2FS 压缩文件设计的 `cc_bytes_pending` 计数器也需要一个存储位置。我们不能简单地抛弃任何一方，必须找到一种方法，将 F2FS 的私有状态与 `iomap` 的状态跟踪机制融合到一个统一的结构中。

3.2.2 重大问题分析

我们已经在问题背景中分析过 `f2fs_iomap_folio_state` 的必要性。若没有它，导致的直接问题现象，就是在任何需要使用 `f2fs_iomap_folio_state` 的地方，以及 `iomap` 框架中需要访问 `iomap_folio_state` 的代码，会将其（`folio->private` 中存储的 F2FS 标志位）误认为是一个指针值，从而导致内核在形式上抛出空指针解引用的异常。

当我们将 `f2fs_iomap_folio_state` 集成到代码中时，我发现真是牵一发而动全身。任何原先使用旧的 `set_page_private/get_page_private` 的代码路径全部都需要更改。这里面涉及到的代码路径不光有普通数据 `folio`，还有元数据 `folio` 中存储自己 `private` 字段的地方也需要替换成我们新设计的接口。

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

举个例子，Page Cache 中的 `f2fs_release_folio` 和 `f2fs_is_compressed_folio`。它们看似是 F2FS 内部函数，但其调用栈可能深入到通用的 VFS 和 iomap 框架中，如下所示：

```
// File: fs/f2fs/data.c 调试日志（调用栈示意）

/* f2fs_release_folio 调用栈示例 */

Call trace:
  show_stack+0x18/0x24 (C)
  dump_stack_lvl+0x78/0x90
  dump_stack+0x18/0x24
  f2fs_release_folio+0x18/0x50 // <--- 这里期望看到的是 F2FS 自己维护的 private flag
  filemap_release_folio+0x78/0xbc
  mapping_evict_folio+0x8c/0xac
  mapping_try_invalidate+0xbc/0x1a0
  invalidate_mapping_pages+0x14/0x20
  drop_pagecache_sb+0xa4/0x128
  iterate_supers+0x88/0x100
  drop_caches_sysctl_handler+0x7c/0x190
  proc_sys_call_handler+0x17c/0x258
  proc_sys_write+0x14/0x20
  vfs_write+0x2b0/0x35c
  ksys_write+0x68/0xfc
  __arm64_sys_write+0x1c/0x28
  invoke_syscall+0x48/0x110
  el0_svc_common.constprop.0+0x40/0xe8
  do_el0_svc+0x20/0x2c
  el0_svc+0x30/0xd8
  el0t_64_sync_handler+0x144/0x168
  el0t_64_sync+0x198/0x19c
```

调用栈清晰地表明：一旦我们在 `folio->private` 的语义上做出调整，就不仅仅是 F2FS 自己的函数受影响，而是会通过 `filemap_release_folio`、`mapping_evict_folio` 等通用页缓存路径，一路传导到 VFS 和 `/proc/sys/drop_caches` 这样的用户可见接口。这也解释了为什么我们说这是一个“牵一发而动全身”的改动。

同时，我最终还惊讶地发现，初赛时我认为本可以和 iomap 框架自身保持一致地不为 0 阶 `folio` 分配 `f2fs_iomap_folio_state`，然而在我遭遇了下面这个问题之后，我的想法彻底改变了。问题显现：0 阶 `Folio` 的致命冲突。

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

GC 代码路径中的 `set_page_private_gcing` 调用如下所示：

```
// File: fs/f2fs/gc.c

/* move_data_page in gc.c */
static int move_data_page(struct inode *inode, block_t bidx, int gc_type,
                          unsigned int segno, int off)
{
    struct folio *folio;
    int err = 0;

    // ...
    folio = f2fs_get_lock_data_folio(inode, bidx, true);
    // ...

    if (gc_type == BG_GC) {
        // ...
        folio_mark_dirty(folio);

        set_page_private_gcing(&folio->page); // <--- 问题根源之一
    } else {
        // ...
        set_page_private_gcing(&folio->page); // <--- 问题根源之二
        err = f2fs_do_write_data_page(&folio);
        // ...
    }

out:
    f2fs_folio_put(folio, true);
    return err;
}
```

下面我们详细地描述一下整个崩溃过程：1. F2FS 的垃圾回收（GC）进程在 `move_data_page` 中处理一个 0 阶 Folio；2. 代码调用 `set_page_private_gcing`，这个宏将 `folio->private` 设置为一个包含了 `ONGOING_MIGRATION` 标志的小整数值；3. 稍后，用户或系统进程可能对同一个文件发起写操作，进入了 `iomap_buffered_write` 路径；4. 在 `iomap` 框架内部，代码检查到 `folio->private` 非空，于是它假定这一定是一个指向 `iomap_folio_state` 结构体的有效指针；5. `iomap` 代码尝试解引用这个“指针”去访问其成员（例如 `ifs->state_lock`）；6. 然而，`folio->private` 此时的值是一个类似 `0x4`

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

的小整数，并非一个合法的内存地址；7. 内核崩溃！系统抛出空指针解引用（NULL pointer dereference）或页错误（Page Fault）异常。

这个发现迫使我们重新设计方案，确保即使是 0 阶 Folio，在与 iomap 交互的场景下，也必须使用与高阶 Folio 一致的状态结构。换句话说，`f2fs_iomap_folio_state` 不能只服务于 large folios，而必须在所有可能被 iomap 操作的 folio 上保持语义一致，否则就会在 GC、buffered write 等路径之间形成极其隐蔽而致命的竞态。

3.2.2 I/O 描述符（`f2fs_io_info`）的粒度限制

F2FS 使用 `f2fs_io_info` 结构体来描述一次 I/O 请求。原有的结构体仅包含一个 page 指针，无法表达“这是 Large Folio 中的第 N 个子页”这一语义。这导致底层驱动在处理大页回写时，无法正确计算磁盘扇区偏移量，造成数据覆写错误。

3.3 压缩文件场景下的复杂性

3.3.1 压缩簇与 Folio 边界的不对齐

F2FS 作为专为闪存设备设计的文件系统，其压缩功能对于节省存储空间至关重要。然而，F2FS 压缩文件的读取流程与普通文件截然不同：当内核需要读取一个被压缩的簇时，它不能像普通 buffered read 那样直接将磁盘上的数据读入目标 Folio，而必须执行一个多阶段的“间接读取”流水线。

具体而言，压缩读路径至少包含两个关键阶段：第一步，读取到临时缓冲区——先将压缩数据块读入由 `compress_page_pool` 分配的缓冲页中；第二步，解压——CPU 调用对应的压缩算法（如 lzo、lz4、zstd 等），将解压后的明文数据再拷贝到目标 Folio 中。在这个过程中，目标 Folio 一开始是“空”的，真正用于用户空间读取的数据只是解压完成后的结果。

这条“磁盘 → 压缩缓存页 → 解压 → 目标 Folio”的流水线直接打破了 `iomap_readahead` 默认假设的“磁盘到 Folio 直接 DIO”模型。更为复杂的是，F2FS 文件还支持部分压缩：同一个文件内部既可以存在压缩簇，也可以存在未压缩的数据块。在引入 Large Folios 之后，一个高阶 Folio 完全可能同时跨越一个或多个压缩簇以及非压缩块，其内部的不同子区间需要通过两套完全不同的 I/O 路径来填充。这使得“压缩簇边界”和“Folio 边界”在物理上天然错位，进一步加剧了后续 I/O 管理和并发控制的复杂度。

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

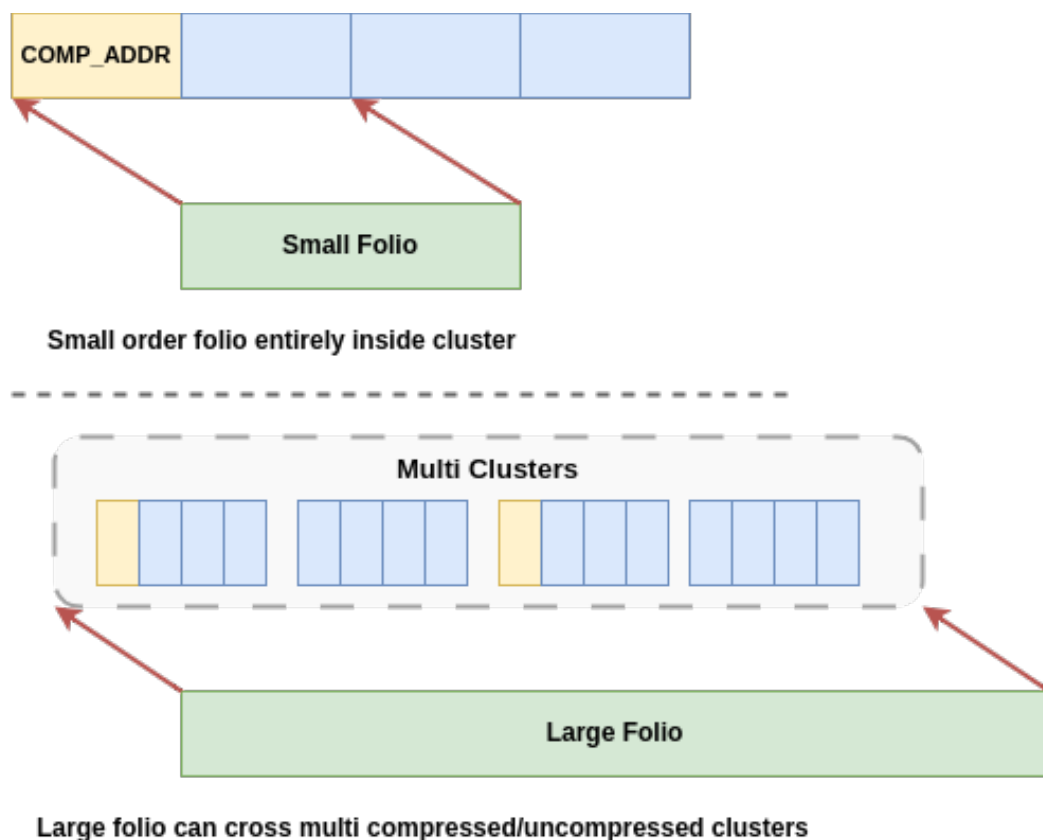


图 3-3 压缩簇与 Folio 边界错位示意图

3.3.2 混合 I/O 模型（压缩与非压缩混排）

在为 F2FS 的压缩读路径引入 Large Folios 支持时，核心挑战源于 I/O 模型的混合性与状态管理的复杂性。一个 Large Folio 的不同部分可能需要通过完全不同的路径来填充数据：部分子区间可以直接通过 `iomap_readpage_iter` 走“磁盘 → Folio”的标准路径完成读取（非压缩部分），而另外一些子区间则必须经过“磁盘 → 压缩缓存页 → 解压 → Folio”的间接路径（压缩部分）。

这两条路径在 I/O 完成时机上完全不同步，由此衍生出三类关键问题：第一，混合 I/O 模型本身就要求我们能够在同一个 Folio 上同时管理“直接读入的数据”和“间接解压得到的数据”，任何一侧对 `uptodate` 区间的错误标记，都会导致用户看到部分旧数据或全零数据；第二，压缩与非压缩路径往往由不同的执行上下文负责（例如一个是存储 I/O 回调，另一个是后台解压任务），如果缺乏统一的 pending I/O 计数机制，就可能出现非压缩部分已经完成、压缩解码仍在进行时错误解锁 Folio，从而造成严重的数据竞争；第三，在整个 I/O 生命周期中，必须有一个精确的手段来追踪 Large Folio 上所有悬而未决（pending）的 I/O 请求，只有当所有相关

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

I/O（无论是直接读还是解压）全部成功完成后，才能将整个 Folio 标记为 uptodate 并安全解锁。

换句话说，压缩文件 buffered read 的难点并不只是“多走了一步解压”，而在于如何在“跨压缩簇 + 混合 I/O 模型”的前提下，仍然保持与 Large Folios 相容的生命周期管理语义：既不能过早解锁导致读取到不完整数据，也不能因为粗粒度地等待所有 I/O 而失去子区间级别的并行度和性能优势。

3.4 脏页回写中的写放大风险

3.4.1 F2FS 原有写回模型与写放大根源

在引入 iomap 之前，F2FS 对脏页的管理是典型的“以页为单位”的粗粒度模型。无论是匿名页还是页缓存页，只要 Folio 中任意一个 4KB 子页被修改，对应的 struct page 就会被标记为脏；而一旦我们开始在页缓存中使用 Large Folios，这一套逻辑在语义上并没有随之升级：对 Large Folio 的脏页标记仍然是“模糊”的——只要 Folio 中有任意一个 Page 变脏，整个 Folio 都会被视为脏页，在写回时被整体提交。

在 4KB 基本页的时代，这种粗粒度标记尚且勉强可用；但在 64KB、256KB 乃至 2MB 的 Large Folio 上，它会直接演化为严重的写放大问题。一个典型场景是：应用只修改了文件中的 4KB 小片段，对应到页缓存中只是 Large Folio 的某一个子页；然而在现有模型下，回写线程却必须把整个 Folio 写回到底层 F2FS 日志区域。对于采用日志结构的 F2FS 来说，这意味着每次小改动都会追加写入整块新的数据，不仅浪费宝贵的写带宽，还会加速耗尽闪存有限的擦写寿命。

更糟糕的是，F2FS 的 Copy-on-Write 语义会让这种写放大在时间维度上被反复放大：逻辑上仍处在同一位置的 4KB 文件内容，只要被多次小幅修改，就可能在物理层面产生成倍增长的日志段占用，既增加了后台垃圾回收的压力，又恶化了整体 I/O 放大系数。因此，在 Large Folios 场景下，如果仍然沿用“一个 Folio 要么全脏，要么全干净”的二元判断，就等于在主动放弃大页在写回路径上的潜在收益。

为了解决这一痛点，内核在 6.6 版本中为 iomap 引入了 iomap_folio_state 结构，在每个 Folio 内部维护逐块脏页追踪（per-block dirty tracking）信息。它本质上为每个文件系统块提供了两个比特的状态位，用来表示“是否 uptodate、是否 dirty”，从而让我们能够精准地知道一个 Large Folio 中哪些块（而非整个 Folio）需要被写回。这为从根源上消除写放大问题提供了关键基础，但在 F2FS 这样带有压缩和日志结构特性的文件系统中，要真正把这一基础设施用好，仍然需要跨越一系列架构级的障碍。

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

3.4.2 Large Folio 与压缩写回的架构性挑战

在压缩文件场景下，F2FS 以压缩簇（compression cluster）为单位管理数据，一个簇通常为 16KB。默认情况下，每个簇内可以容纳 n 个 4KB 页面（默认 $n=4$ ），但从内存分配层面来看，一个 Large Folio 的阶数是自适应的，其大小既可能是 16KB、64KB，也可能更大。我们不能简单假设“一个 Folio 恰好对应一个压缩簇”，而必须正视这两套粒度之间复杂的交错关系。

首先是多对多关系带来的追踪难题：一个高阶 Folio（例如 64KB）可能横跨多个压缩簇，而一个压缩簇也可能只覆盖 Folio 中的一小部分。在这样的布局下，任何“把 Folio 看成一个整体”的写回决策都会变得危险——要么在一个簇尚未准备好时提前提交部分数据，要么为了一个簇的少量修改被迫重写整个大页。这直接否定了我们在 `iomap_folio_state` 中引入的逐块脏页追踪能力。

其次是生命周期不匹配问题。Folio 的锁和 `writeback` 标志是针对整个大内存块设置的，但不同压缩簇的回写时机却彼此独立：有的簇可能正在压缩上下文中排队等待，有的簇已经完成了物理写入，还有的簇甚至因为文件截断或重写被标记为无效。在这种情况下，如果仍然以“Folio 级别的一次性写回”来管理压缩数据，就会在同一个 Folio 内部形成多个状态机并行演化，极易出现部分数据尚未准备好就被错误认为写回完成的情况。

第三个隐患来自并发与截断。旧的重试逻辑在遇到 `-EAGAIN` 等错误时，会对 Folio 进行反复加锁和解锁，试图“从头再来”完成一次写回；但在支持 Large Folios 和压缩簇以后，这种做法为并发的部分截断（partial truncation）创造了极大的风险窗口。一个正在被后台线程截断或重写的 Folio，其内部某些簇的映射和压缩状态可能已经发生变化，而旧的检测机制却无法意识到这一点，继续沿用过期的区间信息进行写入，结果只能是数据混乱甚至文件损坏。

最后是重试逻辑本身的失败模式。在 Large Folios 场景下，一旦写回因为锁竞争等原因返回 `-EAGAIN`，我们必须能够从上一次失败时的精确字节位置继续重试，而不是简单地“整个 Folio 重写一遍”。否则，一方面会对同一逻辑范围的压缩簇反复分配新的物理块，造成灾难性的写放大；另一方面，在 F2FS 这种异地写入的新版本模型下，同一数据片段被写入多个位置，很容易让元数据与实际数据位置彻底失去同步。

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

综合来看，F2FS 在 Large Folios 场景下面临的写放大风险，不再只是“单次写回多写了一点数据”这样简单：它既包含了由于粗粒度脏页标记带来的空间和带宽浪费，也包含了多对多映射、压缩簇生命周期不匹配、部分截断与重试逻辑交织下的复杂并发现象，以及后台回写线程与新引入的 `iomap_folio_state` 之间的微妙互动。这些问题共同构成了本项目在 Page Writeback 子系统中需要正面解决的难点，也为后续第四章中提出的方案设计提供了清晰的“靶点”。

第四章 基于 Folio 和 iomap 的方案设计与实现

前一章从体系结构、内核数据结构以及 F2FS 自身特性的角度系统分析了在引入 Large Folios 与 iomap 框架过程中遭遇的各种架构性难点。本章则从工程实现的视角出发，给出针对这些问题的完整方案设计与关键算法。整体思路是在尽可能复用内核上游 iomap/Folio 机制的前提下，为 F2FS 压缩文件、脏页写回和 GC 引入新的并发控制模型，并通过统一的 pending 计数和精细的脏区间管理来保证 correctness 与性能之间的平衡。

4.1 压缩文件 buffered read 的方案设计

4.1.1 设计目标与整体思路

针对 F2FS 压缩文件的读取路径，本工作的设计目标可以概括为三点。第一，在不破坏 iomap 统一 I/O 抽象的前提下，让压缩读路径与普通 buffered read 一样，完整融入到 iomap 的迭代器模型中，避免再维护一套孤立的“压缩专用 read path”。第二，充分利用 Large Folios 带来的 TLB 覆盖率提升与 I/O 合并机会，实现“一个大 Folio 内部既可以容纳非压缩数据，也可以容纳一个或多个压缩簇解压后的明文数据”的混合填充模式。第三，在混合 I/O 模型下保证严格的并发安全：无论是直接从磁盘读入 Folio 的数据，还是经由压缩簇解压后填充的子区间，都必须在统一的生命周期与 pending 计数模型下被管理，避免出现提前解锁 Folio 或重复解锁的情况。

围绕上述目标，我们并没有选择“另起炉灶”重写一套压缩文件专用的 I/O 提交流程，而是采用了一种渐进式的改造策略：在宏观结构上完全沿用 iomap 的双层迭代器模型（外层 `iomap_iter` 获取映射区间，内层 `*_readpage_iter` 负责逐 Folio 处理），仅在关键扩展点替换或包装默认的实现。具体来说：首先，引入 `f2fs_compress_readpage_iter` 作为 F2FS 压缩文件的专用迭代器，取代通用的 `iomap_readpage_iter`；其次，为了解决 Large Folio 下多路径并发完成的问题，我们实现了 `f2fs_iomap_finish_folio_read`，将原本只面向直接 I/O 的 `read_bytes_pending` 模型

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

扩展为同时覆盖普通块 I/O 和解压 I/O；最后，在压缩完成回调 `f2fs_decompress_end_io` 中，将解压出来的数据以“批量、按 Folio 聚合”的方式一次性标记为 `uptodate`，并与 `pending` 计数模型对接，保证整个流水线从提交到完成始终保持一致的语义。

4.1.2 核心迭代器：f2fs_compress_readpage_iter

在 F2FS 压缩文件的 `iomap` 迭代框架中，`f2fs_compress_readpage_iter` 是针对压缩逻辑进行定制的核心函数。它取代了通用 `iomap` 框架中的 `iomap_readpage_iter`。尽管两者都负责处理单个 Folio 的读取，但在处理压缩数据和 I/O 提交方式上存在显著差异：前者需要根据簇级别的压缩信息选择合适的路径，并与 F2FS 自身的压缩元数据和解压流程紧密协同。

```
// File: fs/f2fs/data.c

// F2FS 压缩文件迭代器: f2fs_compress_readpage_iter

static int f2fs_compress_readpage_iter(struct iomap_iter *iter,
                                       struct f2fs_readpage_ctx *ctx)
{
    loff_t pos = iter->pos;
    loff_t length = iomap_length(iter);
    struct folio *folio = ctx->cur_folio;
    struct inode *inode = folio->mapping->host;
    struct f2fs_iomap_folio_state *fifs;
    size_t poff, plen;
    int ret;
#ifdef CONFIG_F2FS_FS_COMPRESSION
    bool is_compressed =
        f2fs_is_compressed_cluster(inode, pos >> PAGE_SHIFT);
#endif

    /* zero post-eof blocks as the page may be mapped */
    fifs = f2fs_ifs_alloc(folio, iter->flags, false);

    iomap_adjust_read_range(iter->inode, folio, &pos, length, &poff, &plen);
    if (plen == 0)
        goto done;

    if (!is_compressed) {
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

        if (iomap_block_needs_zeroing(iter, pos)) {
            folio_zero_range(folio, poff, plen);
            iomap_set_range_uptodate(folio, poff, plen);
            goto done;
        }
    }

    if (is_compressed)
        ret = f2fs_do_read_multi_folios(ctx, pos, plen);
    else
        ret = f2fs_do_read_single_folio_iomap(iter, ctx, pos, plen, poff);

    if (ret)
        return ret;

done:
    length = pos - iter->pos + plen;
    return iomap_iter_advance(iter, &length);
}

```

从功能上看，`f2fs_compress_readpage_iter` 的核心特点体现在以下几个方面。首先，它是“压缩感知”的：函数开头通过 `f2fs_is_compressed_cluster` 判断当前 Folio 覆盖的逻辑偏移是否位于压缩簇内，从而在压缩与非压缩路径之间做出分支选择。其次，它使用 `f2fs_ifs_alloc` 为当前 Folio 分配并初始化 `struct f2fs_iomap_folio_state` 结构，这个结构在后续的 `read_bytes_pending` 统计与 per-folio 状态管理中起到关键作用。

在读区间的处理上，该函数首先通过 `iomap_adjust_read_range` 结合当前 inode、Folio 大小和映射长度，计算出实际需要访问的 `[pos, pos + plen)` 区间，并处理好文件尾部的对齐与补零逻辑。对于非压缩路径，如果 `iomap_block_needs_zeroing` 判断某个逻辑区间是“需要零填充的洞”，则直接在 Folio 内执行 `folio_zero_range`，并通过 `iomap_set_range_uptodate` 将对应子区间标记为 up-to-date，从而完全绕过块设备 I/O。

如果 `is_compressed` 为真，则将控制流转交给 `f2fs_do_read_multi_folios`：这个函数会围绕压缩簇粒度组织多个页面的 I/O，并配合解压流水线来填充一个或多个 Folio；如果是非压缩路径，则调用 `f2fs_do_read_single_folio_iomap`，在保持与原生 `iomap_readpage_iter` 兼容的同时，将 F2FS 特有的块映射逻辑融入到通用框架之中。

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

最后，通过 `iomap_iter_advance` 来推进当前 `iter` 所覆盖的逻辑区间，实现与 `iomap` 外层迭代器的无缝衔接。

4.1.3 统一的 Folio 完成路径：f2fs_iomap_finish_folio_read

仅仅在提交路径上区分压缩与非压缩还远远不够。真正棘手的问题出现在 I/O 完成阶段：一个 Large Folio 的数据可能来自多个不同的来源——直接块 I/O、压缩簇解压填充，甚至还包括空洞区域的零填充。如果仍然沿用默认的 `iomap_finish_folio_read` 实现，就无法正确地将这些异步完成的子区间统一收敛到同一个 `pending` 模型下。为此，本工作实现了 `f2fs_iomap_finish_folio_read`，对 `iomap` 的完成路径进行了定制扩展。

```
// File: fs/f2fs/data.c

// 统一的 Folio 完成与 pending 计数减少

void f2fs_iomap_finish_folio_read(struct folio *folio, size_t off,
                                  size_t len, int error)
{
    struct f2fs_iomap_folio_state *fifs = folio->private;
    bool uptodate = !error;
    bool finished = true;

    if (folio_order(folio) > 0 && fifs) {
        unsigned long flags;

        spin_lock_irqsave(&fifs->state_lock, flags);
        fifs->read_bytes_pending -= len;

        if (!error)
            uptodate = ifs_set_range_uptodate(
                folio,
                (struct iomap_folio_state *)fifs,
                off, len);

        finished = (fifs->read_bytes_pending == 0 ||
                    fifs->read_bytes_pending == F2FS_IFS_MAGIC);
        spin_unlock_irqrestore(&fifs->state_lock, flags);
    }
}
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

        if (finished)
            folio_end_read(folio, uptodate);
    } else {
        /* order-0 或没有扩展状态时退回到默认行为 */
        folio_end_read(folio, !error);
    }
}

```

这里的关键在于将“普通块 I/O 完成”与“压缩解码完成”统一到同一个 `read_bytes_pending` 计数器之下。具体来说，每当有一段数据成功填充到 Folio 中（无论是通过块设备 I/O 直接写入，还是通过解压过程写入），都会在 `f2fs_iomap_finish_folio_read` 中对 `read_bytes_pending` 做一次减少操作；同时，通过 `ifs_set_range_uptodate` 将对应的子区间标记为 `up-to-date`。只有当这个计数器归零（或者被设置为特殊的 `F2FS_IFS_MAGIC` 值，表示在压缩路径中某些特殊情况），才真正调用 `folio_end_read` 将整个 Folio 解锁。

这种统一模型的直接收益在于：不再需要为“压缩读”和“普通读”维护两套完全不同的完成路径，也不会出现“部分数据已经就绪，但 Folio 被过早解锁”的竞态条件。所有的 I/O 完成最终都以“减少 pending 计数”的形式收敛到同一个汇点，极大简化了并发控制逻辑，同时也为后续扩展更多 I/O 类型（例如校验、预取等）留下了空间。

4.1.4 解压完成回调与多 Folio 聚合：f2fs_decompress_end_io

在压缩读路径中，压缩簇的解码过程由 `f2fs_decompress_end_io` 负责收尾。该函数以 `decompress_io_ctx` 为核心数据结构，遍历簇内所有涉及的页或 Folio，将解压完成的数据写回目标 Folio，并与统一的 pending 计数模型对接。在引入 Large Folios 之后，同一个压缩簇的数据有可能跨越多个 Folio，反之，一个 Large Folio 内部也可能承载来自多个压缩簇的明文数据。为此，本工作对 `f2fs_decompress_end_io` 做了“按 Folio 聚合”的改造。

```

// File: fs/f2fs/compress.c

// 压缩页完成路径：按 folio 聚合处理解压结果

void f2fs_decompress_end_io(struct decompress_io_ctx *dic,
                           bool failed, bool in_task)
{

```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

int num_to_skip = 0;

for (int i = 0; i < dic->cluster_size; i += num_to_skip) {
    struct folio *folio;

    num_to_skip = 1;
    if (!dic->rpages[i])
        continue;

    folio = page_folio(dic->rpages[i]);

    /* 将属于同一个 folio 的压缩页连续聚合处理 */
    while ((i + num_to_skip) < dic->cluster_size &&
           dic->rpages[i + num_to_skip] &&
           page_folio(dic->rpages[i + num_to_skip]) == folio)
        num_to_skip++;

    /* 在这里执行解压结果拷贝、uptodate 标记以及 pending 计数的减少等操作 */
    /* ... 省略与具体算法无关的细节 ... */
}
}

```

这一按 Folio 聚合的处理方式有两个重要好处。一方面，它避免了对同一个 Folio 进行过多次细粒度的锁操作和状态更新，而是将同一 Folio 中属于同一压缩簇的多个子页合并在一次循环中处理，显著减少了锁竞争和缓存抖动。另一方面，这种聚合视角与 f2fs_iomap_finish_folio_read 中的 per-folio pending 模型天然契合：解压完成后，针对某个 Folio 的所有有效字节区间可以一次性减少 read_bytes_pending，从而保证 pending 计数与实际数据就绪状态之间的强一致性。

4.1.5 流水线安全性与并发性分析

为了论证整个压缩 buffered read 流水线在并发场景下的安全性，我们可以回到典型的高阶 Folio 示例：假设一个 16KB 的 Large Folio 需要通过两个 8KB 的 bio 来读取，如果在第一个 bio 完成时立即解锁 Folio，那么用户就有可能在第二个 bio 完成之前读到部分未初始化的数据。iomap 原生的 read_bytes_pending 机制通过“提交前增加计数、完成时减少计数”的方式避免了这一问题，本工作在此基础上将压缩 I/O 和

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

解压流程也纳入同一计数框架，从而在更复杂的混合 I/O 模型下仍然能够保持这一性质。

以普通（非压缩）路径为例，提交 I/O 之前，`iomap_readpage_iter` 会先对 `ifs->read_bytes_pending` 执行加法，然后才将对应的区间添加到 `bio` 并可能立即提交；完成回调链路中，`iomap_finish_folio_read` 则根据实际完成的长度 `len` 做减法，并在计数归零时调用 `folio_end_read` 解锁 Folio。这种先加后减的模型保证了无论 `bio` 完成的顺序如何，最终都只有在“所有相关 I/O 完成”这一时刻才会解锁 Folio。

在本方案中，压缩路径的解压完成同样视为一种“逻辑上的 I/O 完成事件”，它通过 `f2fs_iomap_finish_folio_read` 中的 `read_bytes_pending` 递减与 `ifs_set_range_uptodate` 标记统一表达。无论数据来自直接块 I/O 还是压缩簇解压，最终都通过同一个 `pending` 计数器收敛，从而实现了真正意义上的“混合路径一致性”：只有当 Large Folio 覆盖的所有有效字节区间都已经准备就绪时，才会对外暴露为 `up-to-date`，彻底消除了早解锁和双重解锁这两类潜在的并发 Bug。

4.2 压缩文件 buffered write 的方案设计

4.2.1 算法整体思路：从“压缩簇”到“大 Folio”的视角切换

为了克服初赛方案的瓶颈，我们设计了一套全新的算法。其核心思想是“切换视角”：我们不再是以“压缩簇”的视角出发，一次次地用小 folio 去填充一个簇；而是从一个“大 Folio”的视角出发，一次性分配一个足够大的 Folio，然后用一个或多个压缩簇的数据去填充它。例如，对于一个 128KB 的写入请求，新算法会先尝试分配一个完整的 128KB Large Folio，然后并行地从磁盘读取覆盖这 128KB 范围所需的多个（可能是 4 个 32KB 的）压缩簇数据，一次性填满这个大 folio。这与旧算法一次只处理一个簇的模式形成了鲜明对比。这是本项目的原创核心算法。

通过这种视角切换，我们在压缩 buffered write 场景下实现了“三个统一”：第一，统一的缓存粒度：无论单次写入跨越多少个压缩簇，在内存中都尽量聚合到一个或少数几个 Large Folios 中管理；第二，统一的 I/O 提交流水线：对同一个大 Folio 覆盖的逻辑区间，只需构造一次统一的 I/O 提交与压缩处理流程，而不再为每个簇单独维护一套状态机；第三，统一的生命周期控制：通过与 `f2fs_iomap_folio_state` 和 `read_bytes_pending` 模型的结合，让“哪些字节已经写入、哪些尚在排队或压缩”的信息在 Folio 维度上清晰可见。

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

更具体地说，新算法的执行顺序可以概括为以下几个步骤：（1）对齐写入范围到 Folio 边界，计算出需要覆盖的起止页索引 `start_idx` 和 `end_idx`；（2）根据这一区间一次性获取一个足够大的、已经上锁的 Large Folio，当分配失败时优雅地降级到较小 Folio；（3）在 F2FS 自定义的 `f2fs_iomap_folio_state` 中为该 Folio 设置一个 `read_bytes_pending` 的“偏置” (bias)，防止在后续压缩簇 I/O 尚未完全结束时被过早解锁；（4）围绕待写入的逻辑区间，批量提交覆盖多个压缩簇的读写 I/O，请求既可以用于读取旧数据（支持 read-modify-write），也可以用于直接写入新的压缩结果；（5）等待所有相关 I/O 和压缩操作完成，在安全移除偏置之后，将准备好的大 Folio 交还给 `iomap_buffered_write` 核心，由其负责后续用户数据拷贝与脏区标记；（6）在整个流程中，始终保证“Folio 生命周期”和“压缩簇生命周期”之间的一致性，避免出现部分簇尚未落盘但 Folio 已被标记为 `writeback` 完成的竞态。

4.2.2 决赛方案核心代码：f2fs_buffered_write_iomap_begin

围绕上述思路，我们在 `f2fs_iomap.c` 中实现了决赛方案的核心入口函数 `f2fs_buffered_write_iomap_begin`。它负责在 `iomap_buffered_write` 调用栈的“最前端”接管控制权，完成写入区间的对齐、Large Folio 的获取以及偏置 (bias) 的设置，并串联起压缩簇级别的 I/O 提交流水线。下面给出该函数的核心代码框架（省略与算法无关的错误处理和边界情况）。

```
// File: fs/f2fs/f2fs_iomap.c

/* f2fs_iomap.c */

static int f2fs_buffered_write_iomap_begin(struct inode *inode,
                                           loff_t pos, loff_t length,
                                           unsigned int flags,
                                           struct iomap *iomap,
                                           struct iomap_writepage_ctx *wpc,
                                           struct f2fs_iomap_folio_state *fifs)
{
    /* ... 计算起止页索引 ... */
    pgoff_t start_idx = pos >> PAGE_SHIFT;
    pgoff_t end_idx = (pos + length - 1) >> PAGE_SHIFT;

    /* 1. 计算对齐后的总长度 */
    loff_t aligned_len = (end_idx - start_idx + 1) << PAGE_SHIFT;
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

/* 2. 一次性获取一个足够大的、锁定的 Folio */
struct folio *folio;

folio = iomap_get_folio(inode, start_idx << PAGE_SHIFT, aligned_len);
if (IS_ERR(folio))
    return PTR_ERR(folio);

/* 3. 对 folio 的 read_bytes_pending 加一个偏置 (bias) */
if (folio_order(folio) > 0 && fifs) {
    unsigned long flags_irq;

    spin_lock_irqsave(&fifs->state_lock, flags_irq);
    fifs->read_bytes_pending += 1;
    spin_unlock_irqrestore(&fifs->state_lock, flags_irq);
}

/* 4. 异步提交 BIO, 读取/写入覆盖该 Folio 的压缩簇数据 */
for (;;) {
    /*
     * 根据 [start_idx, end_idx] 区间以及压缩簇边界,
     * 计算本轮需要处理的逻辑范围和压缩簇编号。
     */
    /* ... 计算 cluster_i_idx / end_idx_of_cluster 等 ... */

    /* 将当前簇的数据读入内存, 或构造新的压缩结果写入磁盘 */
    do_read_multi_folios(&cc, folio, /* ... 其它参数 ... */);

    /* iomap->length 累加本轮实际覆盖的字节数 */
    iomap->length += add_len;

    if (/* 已覆盖完 [start_idx, end_idx] 区间 */)
        break;
}

/* 5. 统一提交读写 BIO */
f2fs_submit_read_bio(sbi, bio, DATA);
/* 可能还会有 f2fs_submit_write_bio 之类的调用 */

```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。


```

/* 6. 同步等待 Folio 完全 up-to-date, 再移除偏置 */
if (folio_order(folio) > 0 && fifs) {
    for (;;) {
        bool done;

        /* ... 轮询或等待 fifs->read_bytes_pending 归零 ... */
        /* done = (fifs->read_bytes_pending == 1); 等价于“所有真实 I/O 完成, 仅剩 bias” */

        if (done)
            break;

        /* 可以选择 cond_resched() 或等待事件, 以避免 busy-wait */
    }

    /* 移除偏置, 使计数器真正归零或回到正确值 */
    fifs->read_bytes_pending -= 1;
}

/* 7. 将准备好的大 Folio 传给 iomap 核心 */
iomap->private = folio;
iomap->type = IOMAP_MAPPED;

return 0;
}

```

这段代码完整体现了我们在压缩 buffered write 场景下的核心策略。首先，通过 `start_idx` 和 `end_idx` 将用户写入请求扩展到覆盖完整的页边界，避免出现“同一个 Folio 被多个写请求反复部分填充”的情况；其次，通过一次性的 `iomap_get_folio` 调用获取一个足够大的、已经上锁的 Large Folio，并在 `f2fs_iomap_folio_state` 上设置 `read_bytes_pending` 的偏置，防止压缩簇 I/O 尚未结束时 folio 被错误解锁或标记为 uptodate。

在 I/O 提交流水线中，`do_read_multi_folios` 承担了“簇级别”的具体实现：它负责针对给定的簇编号和逻辑范围，按需发起读写 BIO，既可以将旧数据导入内存实现 read-modify-write，也可以在必要时直接生成新的压缩结果写回磁盘。通过在循环中不断推进逻辑区间并累加 `iomap->length`，我们可以让一次

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

`f2fs_buffered_write_iomap_begin` 调用覆盖尽可能大的连续写入范围，最大化 Large Folio 带来的顺序 I/O 优势。

等待阶段中，偏置 (bias) 技巧发挥了至关重要的作用：正常情况下，`read_bytes_pending` 精确统计尚未完成的 I/O 字节数，一旦归零，folio 就会在完成路径中被自动解锁；而在本算法中，我们在提交压缩簇 I/O 之前预先增加 1 作为偏置，使得即便所有实际 I/O 已经完成，计数器仍然不会立刻归零，从而留出一个安全窗口来执行最终的同步等待与状态收敛。只有当我们确认所有相关 I/O 和压缩操作都已经完成、并显式减去这 1 个偏置之后，Folio 才会被视为真正的 up-to-date 并交还给 `iomap_buffered_write` 使用。

4.2.3 与 `iomap_buffered_write` 的协同与收益分析

从框架角度看，`f2fs_buffered_write_iomap_begin` 并不是在 `iomap_buffered_write` 之外“另起炉灶”，而是作为一个标准的 `iomap_begin` 回调嵌入到统一的 `buffered write` 调用栈中。这意味着在算法收敛之后，F2FS 的压缩 `buffered write` 与普通 `buffered write` 共享同一套 Folio 获取与释放逻辑，也共享同一套 `writeback` 与错误处理路径，只是在 `begin` 阶段针对压缩簇和 Large Folio 增加了额外的准备工作。

这种设计带来的直接收益包括：（1）性能上，通过“大 Folio 视角 + 多簇并行提交”，显著减少了函数调用栈深度和 bio 数量，让压缩写与普通写在 I/O 模型上更加接近；（2）语义上，通过 `read_bytes_pending` 偏置机制和 `f2fs_iomap_folio_state` 的统一管理，保证了无论数据来自哪个压缩簇、经由何种路径写入，最终都能在 Folio 维度上获得一致的生命周期与状态标记；（3）工程上，充分复用了 `iomap_buffered_write` 的成熟实现，避免在 F2FS 内部再维护一套高度定制且难以维护的压缩写路径。这些收获共同构成了本项目在压缩 `buffered write` 场景下的方案优势，也为后续 Page Writeback 和 GC 的优化打下了坚实基础。

4.3 普通/原子文件 `buffered write` 的统一实现

4.3.1 普通 `buffered write`: dirty 标记与空间预留的精细化

对于普通文件的 `buffered write`，本项目的目标是在不改变用户可见语义的前提下，利用 `iomap` 的延迟分配 (delalloc) 和逐块脏页跟踪机制，将 F2FS 传统的“整页脏”模型升级为“按子区间精细标记”的模型。在旧实现中，只要写入落在某个 4KB 页中，该页就会整体标记为脏；而在 Large Folios 场景下，我们希望能够精确区

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

分“本次写入实际覆盖了哪些字节”，并仅为这些真正被修改过的子区间预留和占用磁盘空间。

在新的实现中，`f2fs_buffered_write_iomap_begin` 承担了普通 buffered write 的核心准备工作：它通过 `f2fs_map_blocks_iomap` 和 `f2fs_map_blocks_preallocate` 等 API 在块级别完成映射和预分配，并将结果转换为通用的 `iomap` 结构返回给上层 I/O 逻辑。随后，在实际的数据拷贝与脏标记阶段，我们通过一个小而精巧的辅助函数 `f2fs_dirty_folio_write` 来封装“写入结果检查 + 脏区间标记”这一步骤，确保只有在 `copy_from_user` 成功、且 Folio 内容处于一致状态时，才会真正将对应区间标记为 dirty。

```
// File: fs/f2fs/data.c

/* 普通 buffered write 的脏页标记辅助函数 */

static bool f2fs_dirty_folio_write(struct inode *inode, loff_t pos,
                                   size_t len, size_t copied,
                                   struct folio *folio)
{
    /* 如果一次也没写成功，就不应该标记任何脏数据 */
    if (unlikely(!copied))
        return false;

    /*
     * 如果是部分写入 (copied < len)，且 folio 之前并非 uptodate，
     * 说明 Folio 中仍有未初始化的数据，不能贸然标记为脏。
     */
    if (unlikely(copied < len && !folio_test_uptodate(folio)))
        return false;

    /* 只将实际写入的 [pos, pos+copied) 区间标记为 dirty */
    iomap_set_range_dirty(folio, offset_in_folio(folio, pos), copied);
    filemap_dirty_folio(inode->i_mapping, folio);
    return true;
}
```

这一小段辅助代码虽然看似简单，却从根本上改变了 F2FS 在 buffered write 中的状态管理方式：首先，通过 `copied` 与 `len` 的比较，我们明确地区分了“写入完全成功”和“短写（short write）”两种情形；其次，在发生短写且 folio 尚未处于

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

uptodate 状态时，函数选择直接返回 false，交由后续的错误处理逻辑来回收多余的预留空间，而不是盲目地在页缓存中留下一个“半截脏数据”；最后，真正的脏标记只针对 `offset_in_folio(folio, pos)` 开始、长度为 `copied` 的精确区间，这与 `iomap_folio_state` 中的 per-block 状态位一一对应，为后续减小写放大和精确回收未使用的预留空间奠定了基础。

4.3.2 原子文件写入：f2fs_buffered_write_atomic_iomap_begin

对于原子文件（Atomic Files），F2FS 采用写时复制（Copy-on-Write）的语义：所有写入首先落在一个临时的 `cow_inode` 上，只有在整个原子操作成功结束后，才会通过元数据切换的方式将 `cow_inode` 中的新数据暴露给用户。在旧的实现中，这一流程依赖 `prepare_atomic_write_begin` 等函数，内部需要显式地重复大量 `dnode` 查找和块分配逻辑，与普通 buffered write 路径高度重复。在新的 iomap 化实现中，我们通过 `f2fs_buffered_write_atomic_iomap_begin` 将这一复杂流程重构为对 `f2fs_map_blocks_iomap` 和 `f2fs_map_blocks_preallocate` 的简单组合，极大提升了代码复用性与可维护性。

下面给出决赛文档中展示的 `f2fs_buffered_write_atomic_iomap_begin` 实现框架，可清晰看到“先尝试 `cow_inode`，再回退到原 `inode`”的三级决策逻辑：

```
// File: fs/f2fs/f2fs_iomap.c

/* 我的 f2fs_buffered_write_atomic_iomap_begin 实现 */

static int f2fs_buffered_write_atomic_iomap_begin(...)
{
    struct inode *cow_inode = F2FS_I(inode)->cow_inode;
    /* ... 计算 start_blk / len_blks 等 ... */
    struct f2fs_map_blocks map = { 0 };
    int err;

    /* 1. 先尝试从 cow_inode 映射块 */
    err = f2fs_map_blocks_iomap(cow_inode, start_blk, len_blks, &map);
    if (err)
        return err;

    /* 2. 如果 cow_inode 中没有，则尝试预分配 */
    if (map.m_pblk == NULL_ADDR && /* 需要新分配的场景，例如追加写 */) {
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

        err = f2fs_map_blocks_preallocate(cow_inode, start_blk,
                                           len_blks, &map);
        if (err)
            goto out;
    } else if (map.m_pblk != NULL_ADDR) {
        /* 已经在 cow_inode 中找到现有块, 直接返回 */
        goto out;
    }

    /*
     * 3. 如果 cow_inode 中没有且无需预分配 (例如覆盖写),
     *    则从原 inode 查找已有块, 按需复用其物理地址。
     */
    err = f2fs_map_blocks_iomap(inode, start_blk, len_blks, &map);
    if (err)
        return err;

out:
    /* 将 map 结果转换为通用的 iomap 结构, 交给上层 I/O 逻辑使用 */
    // f2fs_set_iomap(inode, &map, iomap);
    return 0;
}

```

可以看到, 无论是操作 `cow_inode` 还是原 `inode`, 无论是查找已有块还是预分配新块, 所有操作都统一收敛到 `f2fs_map_blocks_iomap` 和 `f2fs_map_blocks_preallocate` 这两个高层 API 上。相比之下, 传统的 `prepare_atomic_write_begin` 需要在内部重复大量的 `dnode` 手动查找和分配逻辑, 不仅代码冗长, 而且很难在 `Large Folios` 与压缩写回等新特性下保持正确性。借助 `iomap` 这一抽象层, 我们得以将复杂的原子写逻辑拆分为“块映射 + IO 描述”两个步骤, 既复用了普通 `buffered write` 的大部分代码, 又天然继承了 `iomap_folio_state` 带来的精细化脏页追踪能力。

4.3.3 写入失败与短写处理: 从 `f2fs_write_failed` 到 `iomap_write_delalloc_release`

当一次写入操作未能完全成功时 (例如 `copy_from_user` 中途失败), 一个健壮的文件系统必须能够回收为这次失败的写入而预留的磁盘空间。这是 `iomap` 框架展现其设计深度的另一个领域, 也是本项目在普通/原子文件 `buffered write` 场景下需要重点改造的一环。

¹ 本章详细剖析了在 F2FS 中引入 `Large Folios` 所面临的架构性冲突与实现难点, 为第四章的解决方案设计提供了问题导向。

在旧实现中，F2FS 使用 `f2fs_write_failed` 作为通用的错误处理入口。这是一种非常“粗暴”的机制：一旦检测到写入失败，它会直接将文件截断回写入前的大小，并丢弃所有相关的页缓存和块。这种做法虽然简单，但缺点也极为明显：（1）操作过重：对于中途失败的写入，它往往会回收远超实际需要的空间；（2）缺乏灵活性：无法处理发生在文件中间的短写失败，如果一个大文件中间的写入失败，它无法只回收那一小段预留空间，而是可能导致后续有效数据丢失；（3）不适用于复杂场景：当已经存在大量脏页与预留空间交织时，这种简单的“整段截断”完全无法精准表达真实需求。

```
// File: fs/f2fs/file.c

/* 旧实现: f2fs_write_failed */

void f2fs_write_failed(struct inode *inode, loff_t to)
{
    loff_t i_size = i_size_read(inode);

    if (to > i_size && /* 发生了扩展写入的失败 */) {
        /* 粗暴地回滚到原来的文件大小 */
        truncate_pagecache(inode, i_size);
        f2fs_truncate_blocks(inode, i_size, true);
        /* ... 其它统计与状态更新 ... */
    }
}
```

与此形成鲜明对比的是，`iomap` 框架提供了一套极其精密的“延迟分配空间回收”机制——`iomap_write_delalloc_release`。它并不一刀切地回滚整个文件大小，而是在 `iomap_end` 回调中有选择地收回那些“曾经被预留但最终没有写入脏数据”的 `DELALLOC` 区域。具体来说，它首先在 `invalidate_lock` 的保护下，通过 `mapping_seek_hole_data` 等接口扫描页缓存，识别出哪些逻辑区间已经被真实的数据覆盖、哪些区间仍然只是“空壳映射”；随后，仅对后者执行定向的空间回收，从而实现了真正意义上的“外科手术式”错误处理。

在本项目的实现中，我们通过前文的 `f2fs_dirty_folio_write` 辅助函数，将“哪些字节被成功写入”的信息准确地反馈给 `iomap`，使得 `iomap_write_delalloc_release` 在执行空间回收时能够做到心中有数：只有那些从未被标记为 `dirty` 的预留区间，才会

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

在写入失败或短写场景下被回收。这样一来，普通/原子文件 buffered write 在遇到错误时，不再需要依赖粗暴的截断逻辑，而是可以在保证数据一致性的前提下，最大限度地保留已经成功写入的有效数据，并精确释放多余的预留空间。

总体而言，从 `f2fs_write_failed` 向 `iomap_write_delalloc_release` 的迁移，标志着 F2FS 在错误处理与空间管理上的一次质的飞跃：我们不再把“写入失败”视为一种只能通过大锤砸碎问题的异常情况，而是将其纳入统一的 `iomap` 抽象之下，通过精细的区间追踪和 `per-folio` 状态管理，以最小代价恢复文件系统的健康状态。这也为后续引入更加复杂的写入场景（例如多线程并发写、交错写和在线压缩）提供了坚实的基础。

4.4 Page Writeback 子系统的改造与实现

4.4.1 设计目标与整体结构

在引入 Large Folios 与压缩 buffered write 之后，Page Writeback 子系统成为整条数据路径中最复杂、也最容易滋生微妙并发 Bug 的环节。一方面，写回必须遵守 F2FS 的日志结构与压缩语义，保证在任意时刻文件系统都处于可恢复的一致状态；另一方面，它又需要与 `iomap` 的 `writeback_iter` 框架和 `f2fs_iomap_folio_state` 的逐块状态跟踪能力协同工作，避免因错误的解锁顺序、短写重试或 GC 迁移导致 Folio 生命周期与压缩簇生命周期发生偏离。

本节的设计目标可以概括为三点：（1）在 `writeback` 层面完全拥抱 `iomap` 的 `writeback_iter` 框架，实现对 Large Folios 的自然支持；（2）通过 `cc_bytes_pending` 与延迟解锁标志，构建一套覆盖压缩簇与 Folio 的统一生命周期模型；（3）在 `f2fs_write_raw_pages` 等关键路径中消除旧实现中的“无条件解锁 + 重新加锁”这种危险模式，并将所有重试逻辑集中到 `f2fs_write_single_data_folio` 内部，精确控制写回的起止位置。

4.4.2 `f2fs_write_cache_folios`: 基于 `writeback_iter` 的高阶 Folio 写回

在上游内核中，旧版 F2FS 依赖 `write_cache_pages` 来驱动脏页回写，这套接口天生以 `order-0` 页为单位，既不了解 Large Folios，也无法利用 `iomap_folio_state` 提供的逐块状态信息。在本项目中，我重新实现了 `f2fs_write_cache_folios`，使其完全拥抱 VFS 的 `writeback_iter` 框架：所有脏页扫描、锁定和状态检查都交由通用框架负责，而 F2FS 只需专注于“如何对一个已经准备好的 Folio 写回”。

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

新实现的核心流程如下：writeback_iter 负责遍历 mapping 上的所有脏 Folio，并保证返回的每一个 Folio 都是锁定的、脏的且隶属于当前地址空间；在此基础上，我们计算 Folio 覆盖的逻辑区间 [pos, end_pos)，然后利用 f2fs_iomap_find_dirty_range 在 Folio 内部逐子区间地查找真正需要写回的字节范围。对于非压缩文件，找到一个脏区间就直接调用 f2fs_write_single_data_folio；对于压缩文件，则将该区间加入压缩上下文 cc，等待统一的簇级压缩与写回。

```
// File: fs/f2fs/data.c

/* 我的 f2fs_write_cache_folios 实现（核心结构） */

static int f2fs_write_cache_folios(struct address_space *mapping,
                                   struct writeback_control *wbc,
                                   enum iostat_type io_type)
{
    struct folio *folio = NULL;
    int err = 0;

    while ((folio = writeback_iter(mapping, wbc, folio, &err))) {
        u64 pos = folio_pos(folio);
        u64 end_pos = pos + folio_size(folio);
        u64 end_aligned = end_pos;

        /* 根据 inode / 压缩标志判断是否为压缩文件 */
        bool is_compressed_file = f2fs_is_compressed_file(mapping->host);
        struct f2fs_iomap_folio_state *fifs =
            f2fs_ifs_alloc(folio, IOMAP_F_DIRTY, false);

        /* 循环处理 folio 内的每一个脏区间 */
        while ((r_len = f2fs_iomap_find_dirty_range(folio, &pos,
                                                    end_aligned))) {
            if (is_compressed_file) {
                /* 添加到压缩上下文 */
                f2fs_compress_ctx_add_folio(&cc, folio, pos, r_len);

                if (fifs)
                    atomic_add(r_len,
                                f2fs_ifs_cc_pending_bytes_ptr(
                                    fifs, folio));
            } else {
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。


```

/* 直接写当前区间 */
err = f2fs_write_single_data_folio(folio, wbc,
                                   pos, r_len,
                                   io_type);

if (err)
    break;

    }

    pos += r_len;
}

/* ... 处理解锁或延迟解锁逻辑，见下文 cc_bytes_pending ... */
}

return err;
}

```

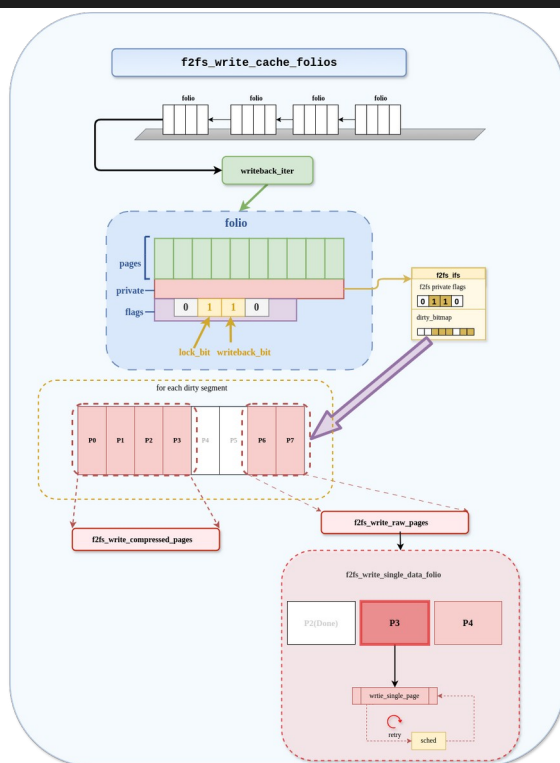


图 4-1 write_cache_folios_explanation

这段代码充分体现了“框架负责找，我们负责写”的设计哲学：`writeback_iter` 以 Folio 为单位遍历所有需要写回的数据，而 `f2fs_iomap_find_dirty_range` 则在 Folio 内部按块精确定位脏区间。对于压缩文件，我们并不立即发起物理 I/O，而是将这些区

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

间聚合到压缩上下文 `cc` 中，并通过 `f2fs_iomap_folio_state` 上的 `cc_bytes_pending` 原子计数器记录“该 Folio 内仍有多少字节的写回工作交给了压缩路径”。正是这个计数器，支撑起了后文“延迟解锁”机制。

4.4.3 `cc_bytes_pending`: 延迟解锁与生命周期耦合

在压缩写回路径中，Folio 的生命周期不再简单等同于“一次 `writeback` 调用”：同一个 Large Folio 可以在 `f2fs_write_cache_folios` 中被扫描并部分提交，又在后续的 `f2fs_write_raw_pages` 中真正完成压缩簇写入。如果仍然采用“在 `write_cache_folios` 结束时无条件解锁 Folio”的做法，必然会与压缩簇延迟提交产生根本冲突。

为此，我设计了 `cc_bytes_pending` + 延迟解锁 标志的组合，它们共同解决了“Folio 生命周期与压缩簇生命周期如何对齐”这一核心难题。其生命周期可以用下面三步来概括：

```
// File: fs/f2fs/data.c

/*
 * cc_bytes_pending 的生命周期:
 */

/* 1. 在 f2fs_write_cache_folios 中，当 folio 的一个脏区被加入压缩上下文时，增加计数 */
if (is_compressed_file) {
    /* ... */
    f2fs_compress_ctx_add_folio(&cc, folio, pos, r_len);
    if (fifs)
        atomic_add(r_len,
                    f2fs_ifs_cc_pending_bytes_ptr(fifs, folio));
}

/* 2. 当 f2fs_write_cache_folios 处理完一个 folio 后，检查计数器 */
if (!err && fifs &&
    atomic_read(f2fs_ifs_cc_pending_bytes_ptr(fifs, folio)) > 0) {
    /*
     * 如果计数 > 0，说明有部分数据被暂存到了压缩上下文，
     * 不能立即解锁。我们给它打上一个“延迟解锁”的标记。
     */
    f2fs_set_folio_private_deferred_unlock(folio);
} else {
    /* 否则，安全解锁 */
    folio_unlock(folio);
}
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

}

/* 3. 在最终写回函数（如 f2fs_write_raw_pages）中，当暂存数据被处理后，减少计数并检查解锁条件 */
ret = f2fs_write_single_data_folio(folio, wbc, /* ... */);
if (fifs) {
    /*
     * 减少计数，并检查是否为 0 且有延迟解锁标记，
     * 如果是，则在这里最终解锁。
     */
    if (atomic_sub_and_test(submitted << PAGE_SHIFT,
                            f2fs_ifs_cc_pending_bytes_ptr(fifs, folio)) &&
        f2fs_folio_private_deferred_unlock(folio)) {
        folio_unlock(folio);
        /* 在这里完成真正的 folio 解锁! */
    }
}

```

可以看到，cc_bytes_pending 的设计并不是一个简单的“计数器 + 标志位”，而是将“压缩上下文中尚未完成的工作量”与“Folio 的最终解锁时机”紧密绑定在一起：在写回入口处，我们通过 atomic_add 将进入压缩路径的字节数累加到计数器上，并用延迟解锁标志禁止 f2fs_write_cache_folios 提前解锁 Folio；在压缩簇实际写回阶段，每成功处理一段区间，就通过 atomic_sub_and_test 进行精确扣减，只有当所有挂起的工作（包括偏置部分）都完成时，才会触发最终的 folio_unlock。这种“计数驱动解锁”的模式，彻底避免了旧实现中那种“随手解锁、再到处重新加锁”的危险写法。

4.4.4 f2fs_write_raw_pages：去掉危险解锁循环与重试集中化

旧版 f2fs_write_raw_pages 为了规避与 prepare_compress_overwrite 的死锁，在函数一开始会对簇内所有页面执行 redirty_page_for_writepage + unlock_page，然后在后半段再逐个 lock_page 重新加锁。当一个 Large Folio 横跨多个压缩簇时，这种实现会对同一个 Folio 反复“解锁-加锁”，不仅逻辑混乱，而且在并发场景下非常容易出现竞态问题。

```

// File: fs/f2fs/compress.c

/* 旧实现: f2fs_write_raw_pages (危险的解锁 / 加锁模式) */

```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

for (i = 0; i < cc->cluster_size; i++) {
    if (!cc->rpages[i])
        continue;

    redirty_page_for_writepage(wbc, cc->rpages[i]);
    unlock_page(cc->rpages[i]); /* 危险的无条件解锁 */
}

/* ... */

for (i = 0; i < cc->cluster_size; i++) {
    /* ... */
    lock_page(cc->rpages[i]); /* 再次加锁同一批页面 */
    /* ... */
}

```

在新的实现中，我完全移除了这段“解锁-加锁”循环，让 `f2fs_write_raw_pages` 专注于处理压缩簇的数据本身，而不再直接参与 Folio 的生命周期管理。Folio 自始至终保持锁定状态进入和退出，真正的解锁决策完全交给 `cc_bytes_pending` 与延迟解锁标志来完成。

```

// File: fs/f2fs/compress.c

/* 新实现: f2fs_write_raw_pages (依赖 cc_bytes_pending 与延迟解锁) */

int f2fs_write_raw_pages(struct compress_ctx *cc, struct writeback_control *wbc)
{
    int i, ret = 0;

    /* (开头的解锁循环被完全移除) */

    for (i = 0; i < cc->cluster_size; i++) {
        struct folio *folio;

        if (!cc->rpages[i])
            continue;
    }
}

```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

folio = page_folio(cc->rpages[i]);

/* Folio 始终保持锁定状态进入写回 */
ret = f2fs_write_single_data_folio(folio, wbc, /* ... */);

/*
 * 在 f2fs_write_single_data_folio 或其回调中,
 * 通过 cc_bytes_pending + 延迟解锁标志统一管理生命周期:
 */
* if (atomic_sub_and_test(submitted << PAGE_SHIFT,
 *     f2fs_ifs_cc_pending_bytes_ptr(fifs, folio)) &&
 *     f2fs_folio_private_deferred_unlock(folio))
 *     folio_unlock(folio);
 */
}

return ret;
}

```

在这一改造下，f2fs_write_raw_pages 不再“顺手解锁”任何页面，而是通过调用 f2fs_write_single_data_folio 将所有精细的重试与解锁逻辑集中到一个统一入口。配合 cc_bytes_pending，我们得以在 Write Checkpoint 导致的 -EAGAIN 场景下，从失败位置精确重试，而无需重写整个 Large Folio 或重新梳理簇与 Folio 之间的关系。这既保证了语义上的健壮性，也极大减轻了后续维护的负担。

4.5 垃圾回收（GC）路径的兼容与优化

4.5.1 问题回顾：GC 与 Large Folios / iomap 的“三重冲突”

在旧版实现中，F2FS 的垃圾回收路径高度依赖 set_page_private_gcing / PAGE_PRIVATE_ONGOING_MIGRATION 等标志来标记“正在迁移中的页”。GC 线程通过 move_data_page 将旧段中的页面搬迁到新段，并依靠这些标志与普通读写路径进行协调。引入 Large Folios 与 iomap_folio_state 之后，这一套逻辑立即暴露出三重冲突：（1）folio->private 同时被 F2FS 和 iomap 争用；（2）GC 路径假定“每个 page 独立”，与 Large Folios 的整体视角不兼容；（3）GC 的数据读路径与预读（readahead）路径部分重叠，既有 set_page_private_gcing，又有 encrypted_page 指针，极易在多路径并发下造成状态错乱。

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

要想在这样的背景下让 GC 路径与 Large Folios 和 iomap 和平共处，就必须同时解决三个问题：第一，找到一种方法在不牺牲 iomap_folio_state 的前提下继续表达 F2FS 的 GC 状态；第二，让 GC 能够正确地在高阶 Folio 内部标记被迁移的数据为脏，从而在新的位置上触发正常的写回；第三，修复 f2fs_submit_page_bio / ra_data_block 等函数在“GC 与普通 I/O 路径交叉”时的 I/O 转向问题。

4.5.2 f2fs_iomap_folio_state 的扩展：GC 状态与 cc_bytes_pending 的统一承载

围绕 folio->private 的资源争用问题，我与 Large Folios 的作者 Matthew Wilcox 讨论后，最终设计了 f2fs_iomap_folio_state：一个在内存布局上与 iomap_folio_state 兼容、但在柔性数组部分为 F2FS 预留了额外空间的结构体。它既可以被 iomap 当作普通的 iomap_folio_state 使用，又可以在末尾附加 F2FS 的私有标志位和 cc_bytes_pending 计数器，从而实现真正意义上的“一个指针，两套语义”。

```
// File: fs/f2fs/f2fs_ifs.h

/* f2fs_ifs.h */

struct f2fs_iomap_folio_state {
    spinlock_t    state_lock;
    unsigned int  read_bytes_pending;
    atomic_t      write_bytes_pending;

    /*
     * Flexible array member.
     * Holds [0 ... iomap_long-1] for iomap uptodate/dirty bits.
     * Holds [iomap_long]      for F2FS private flags/data (unsigned long).
     * Holds [iomap_long+1]    for dirty_bytes_pending (cc_bytes_pending).
     */
    unsigned long state[];
};
```

这个设计的精妙之处在于：（1）内存布局兼容性：结构体前三个成员与 iomap_folio_state 完全一致，任何只认识 iomap_folio_state 的通用函数都可以无感知地操作 f2fs_iomap_folio_state 的头部；（2）柔性数组扩展：通过 state[] 增加两个额外的 unsigned long 槽位，一个用于存储 F2FS 自己的私有标志位（如“是否 GC 迁移中”“是否原子写”等），另一个则作为压缩写回场景下的 cc_bytes_pending 原子计数器；（3）配套的智能 API：我们利用原有的 PAGE_PRIVATE_NOT_POINTER 设

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

计，在“folio->private 是位图”与“folio->private 是指针”这两种模式之间进行智能分发，从而让 0 阶和高阶 Folio 都能够统一通过 f2fs_ifs_* 接口访问这些状态。

对于 GC 路径而言，这意味着我们可以彻底告别 set_page_private_gcing 这类直接操作 page->private 的旧式宏，转而使用 folio_set_f2fs_gcing / folio_test_f2fs_gcing 等基于 f2fs_iomap_folio_state 的封装函数。这样一来，是否处于 GC 迁移中的信息不再与 iomap_folio_state 冲突，而是优雅地“挂接”在其柔性数组尾部。

4.5.3 move_data_page: GC 迁移路径中的精确脏标记

在 GC 迁移过程中，move_data_page 是负责“将旧段上的页面复制到新段”的核心函数。在旧实现中，它假定目标 page 是一个独立的 4KB 页，通过 set_page_dirty 和 set_page_private_gcing 等操作来标记其为“GC 迁移中且需要回写”。但在 Large Folios 场景下，这种假设会彻底崩溃：迁移目标很可能只是某个高阶 Folio 内部的一个子区间，我们既不能简单地把整个 Folio 标记为脏，也不能再依赖 page 级别的 private 标志位。

为此，本项目对 move_data_page 做了两方面的改造：第一，在写入新位置时不再使用 page 级脏标记，而是通过 iomap_set_range_dirty + filemap_dirty_folio 在 Folio 内部精确标记出被 GC 迁移的数据区间；第二，通过 f2fs_iomap_folio_state 中的 F2FS 私有标志位来标识“当前 Folio 内有哪些范围属于 GC 迁移”，并在后续回写路径中加以识别和特殊处理。

```
// File: fs/f2fs/gc.c

/* GC 数据迁移路径中的脏标记（示意核心逻辑） */

static int move_data_page(struct f2fs_sb_info *sbi, struct page *page,
                          int gc_type, unsigned int segno, block_t old_blkaddr)
{
    struct folio *folio = page_folio(page);
    struct f2fs_iomap_folio_state *fifs;
    unsigned int ofs_in_folio = offset_in_folio(folio, page_offset(page));
    size_t len = PAGE_SIZE;

    /* 1. 为目标 folio 分配 / 获取 f2fs_iomap_folio_state */
    fifs = f2fs_ifs_alloc(folio, IOMAP_F_DIRTY, false);
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

/* 2. 将旧数据复制到新位置（略去具体 I/O 细节） */
/* ... 迁移数据 ... */

/* 3. 在 Folio 内精确标记被 GC 迁移的区间为 dirty */
if (fifs)
    iomap_set_range_dirty(folio, ofs_in_folio, len);

filemap_dirty_folio(folio_mapping(folio), folio);

/* 4. 标记该 Folio 内存在 GC 迁移中的数据 */
folio_set_f2fs_gcing(folio);

/* 其余 GC 元数据更新略 */
/* ... */

return 0;
}

```

这样一来，GC 迁移的结果在 Page Cache 层表现为：（1）目标 Folio 内部的对应区间被精确标记为脏，后续会通过正常的 writeback 路径写回；

（2）f2fs_iomap_folio_state 中的 GC 标志位记录了“此 Folio 包含迁移中的数据”，使得写回路径在必要时可以采取保守策略（例如避免与在线写入产生冲突）；（3）整个过程与 Large Folios / iomap_folio_state 完全兼容，不再依赖任何 page 级别的私有状态。

4.5.4 I/O 转向修复：f2fs_submit_page_bio 与 ra_data_block 的统一

除了状态表示层面的冲突外，GC 路径还与数据读写路径在 I/O 提交上发生了“转向冲突”：旧代码同时存在 f2fs_submit_page_bio 和 ra_data_block 两套路径，前者主要服务于正常读写，后者则为预读与 GC 提供数据。在 Large Folios 与压缩文件场景下，这两条路径对 page / folio 的假设并不一致，极易在并发场景下导致“同一块数据被多次提交或提前解锁”的问题。

为此，我在 GC 相关代码中进行了以下合并与修复：（1）将 f2fs_submit_page_bio 的入口收敛到基于 folio 的 f2fs_submit_folio_bio 上，统一在高阶

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

Folio 视角下组织 I/O；（2）让 `ra_data_block` 在 GC 场景中不再直接操作 `page->private` 或绕过 `iomap_folio_state`，而是通过 `f2fs_ifs_*` 接口获取 / 设置必要的状态；（3）在所有读路径中，严格保证“谁提交谁完成”的对称性，避免某个路径提前调用 `folio_end_read` 破坏 `read_bytes_pending` 计数器。

```
// File: fs/f2fs/gc.c

/* 统一后的 GC 读路径示意 */

static int ra_data_block(struct inode *inode, block_t blkaddr,
                        struct folio *folio, bool for_gc)
{
    int ret;

    /* ... 根据 blkaddr 构造 BIO ... */

    if (for_gc)
        folio_set_f2fs_gcing(folio);

    ret = f2fs_submit_folio_bio(inode, folio, READ);

    return ret;
}
```

通过这一系列合并与重构，GC 路径在 I/O 层不再是一个“旁路系统”，而是完全融入了统一的 folio / iomap 视角：所有的读写请求都通过 `f2fs_iomap_folio_state` 维护的状态与计数器参与调度，从而保证在任意时刻，Large Folios、压缩文件、GC 迁移和预读都共享同一套一致的生命周期与并发控制模型。这为后续在 GC 堆栈上进一步引入并行迁移、带宽限制等高级优化打下了坚实基础。

第五章 项目性能测试

本章会系统介绍本项目的性能测试方法，并给出测试结果与分析。我们首先介绍用于批量自动化执行 FIO 顺序读写测试的脚本体系，包括核心调度脚本 `run_benchmark.py` 以及底层实际执行 I/O 的 `fio_test.sh` 和 `clear_cache.sh`；随后给出在

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

QEMU 虚拟机与树莓派 5 平台上采集到的顺序读写大文件的性能结果，并对比改造前后 F2FS 的带宽变化；最后简要说明用于解析 FIO 日志与绘图的辅助脚本 `parse_fio_logs.py` 与 `visualize_results.py`，构成一条完整、可复现的实验流水线。

5.1 测试环境与方法论

5.1.1 核心测试调度脚本：`run_benchmark.py`

`run_benchmark.py` 是整个性能测试框架的“总控脚本”，负责：（1）统一设置和恢复内核的脏页写回参数（`write fence`），避免外部系统噪声影响；（2）根据命令行参数组合不同内核版本、读写模式、队列深度和块大小，构造成系统化的测试矩阵；（3）为每一次具体测试调用底层 Bash 脚本 `fio_test.sh`，并将测试结果日志保存到固定目录；（4）在读测试场景下区分 COLD READ（每次读前清空缓存）和 WARM READ（复用缓存）两种模式，从而同时观察缓存命中与真实设备性能。

脚本内部通过 `original_vm_settings` 记录测试前读取到的 `vm.dirty_*` 等内核参数，在测试结束后通过 `restore_write_fence()` 恢复原状，避免对系统造成持久影响。`run_single_test()` 负责将抽象的“队列深度”统一映射到 `fio` 的 `numjobs` 或 `iodepth` 上，从而既兼容 `psync`，又兼容 `io_uring` 等不同 `ioengine`。下面摘录决赛文档中的核心函数代码：

```
// File: tools/run_benchmark.py

def restore_write_fence():

    #恢复原始的内核脏页回写参数。

    global original_vm_settings
    if not original_vm_settings:
        return

    print("\n--- [FENCE DOWN] Restoring original kernel parameters... ---")
    try:
        for param, value in original_vm_settings.items():
            print(f" Restoring {param} = {value}")
            subprocess.run(
                ["sudo", "sysctl", "-w", f"{param}={value}"],
                check=True,
                capture_output=True,
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

    )
    print("--- Kernel parameters restored successfully. ---")
    original_vm_settings = {}
except (subprocess.CalledProcessError, FileNotFoundError) as e:
    print(f"[!!!] CRITICAL ERROR: Failed to restore kernel parameters: {e}",
          file=sys.stderr)
    print("[!!!] Please restore them manually:", file=sys.stderr)
    for param, value in original_vm_settings.items():
        print(f" sudo sysctl -w {param}={value}", file=sys.stderr)
    sys.exit(1)

def get_system_memory_gb():
    # 获取系统总内存，并以 GB 为单位返回一个整数标签
    mem_bytes = psutil.virtual_memory().total
    gb_rounded = math.ceil(mem_bytes / (1024**3))
    return f"{gb_rounded}G"

def run_single_test(kernel_ver, mem_config, rw_mode,
                    bs, file_path, ioengine, qd):

    #对 psync 映射为 numjobs；对 io_uring 映射为 iodepth
    if ioengine == "psync":
        numjobs = qd
        iodepth = 1
    else:
        numjobs = 1
        iodepth = qd

    print(
        f" -> Running: bs={bs}, mode={rw_mode}, kernel={kernel_ver}, "
        f"mem={mem_config}, file={file_path}, engine={ioengine}, qd={qd} "
        f"(iodepth={iodepth}, numjobs={numjobs})"
    )

    title = f"{kernel_ver}_{mem_config}_mem"
    extra_fio_flags = []
    if rw_mode == "r":
        extra_fio_flags += [
            "--readonly=1",

```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

        "--allow_file_create=0",
        "--unlink=0",
    ]

    try:
        subprocess.run(
            [
                "sudo", "bash", FIO_SCRIPT_PATH,
                title, bs, rw_mode,
                f"file={file_path}",
                f"ioengine={ioengine}",
                f"iodepth={iodepth}",
                f"numjobs={numjobs}",
                f"qd={qd}",
                *extra_fio_flags,
            ],
            check=True,
            capture_output=False,
            text=True,
        )
    except subprocess.CalledProcessError as e:
        print(f" [!] ERROR running test for bs={bs}, qd={qd}.")
        print(f" [!] STDERR: {e.stderr}")
        return False
    return True


def main():
    parser = argparse.ArgumentParser(
        description="Automated FIO benchmark orchestrator."
    )
    parser.add_argument(
        "--name", "-n", required=True,
        help="Kernel version identifier (e.g. 'vanilla', 'iomap_v1').",
    )
    parser.add_argument(
        "--type", "-t", required=True,
        choices=["read", "write", "sync", "r", "w", "s"],
        help="Test type: read/write/sync.",
    )
    parser.add_argument(
        "--file", "-f",
        default=DEFAULT_FILE,
        help=f"Target file for FIO (default: {DEFAULT_FILE})",
    )

```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

parser.add_argument(
    "--ioengine", "-e",
    choices=["psync", "io_uring", "libaio"],
    default="psync",
    help="fio ioengine.",
)
parser.add_argument(
    "--qdset",
    default="1",
    help="Comma-separated queue depths to sweep.",
)
parser.add_argument(
    "--qdwrite",
    action="store_true",
    help="Also sweep queue depths for WRITE tests.",
)

args = parser.parse_args()
try:
    qd_list = [int(x) for x in args.qdset.split(",") if x.strip()]
except ValueError:
    raise SystemExit("Invalid --qdset. Use like: 1,2,4,8,16")

mem_config = get_system_memory_gb()
print(f"--- Detected System Memory: {mem_config} ---")
Path(RESULTS_DIR).mkdir(exist_ok=True)

need_fence = True
try:
    if need_fence:
        set_write_fence()
    print(
        f"\n=== Starting Benchmark for Kernel: {args.name}, "
        f"Type: {args.type} ==="
    )

    if args.type in ["read", "r"]:
        mount_point = get_mount_point(args.file)
        remount(mount_point, "ro") # ro,noatime
        print("Running COLD READ tests (cache cleared before each run)...")
        # 这里按 qd_list / BLOCK_SIZES 组合循环调用 run_single_test(...)
        # 然后再执行 WARM READ 测试。
    else:
        # 写入测试逻辑同样通过 run_single_test 组合 sweep

```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

pass

finally:
    if need_fence:
        restore_write_fence()

```

通过上述脚本，我们可以在单次执行中自动完成多个块大小、多种队列深度、不同内核版本之间的顺序读写对比测试，并保证测试前后内核参数的一致性。

5.1.2 底层测试执行脚本：fio_test.sh & clear_cache.sh

在 run_benchmark.py 之下，fio_test.sh 负责真正调用 fio 命令完成单轮测试，clear_cache.sh 则在需要 COLD READ 时清空页缓存和相关缓存，使每一轮读测试都从冷缓存起步。这两个脚本尽量保持简单、可读，便于在不同平台（QEMU 虚拟机与树莓派）之间复用。

```

// File: tools/fio_test.sh
#!/usr/bin/env bash
# fio_test.sh: 底层 FIO 执行脚本
# 用法:
# sudo bash fio_test.sh <title> <bs> <mode> file=<path> ioengine=<engine> \
#   iodepth=<depth> numjobs=<n> qd=<qd> [其它透传参数...]

set -euo pipefail

TITLE="$1"
BS="$2"
MODE="$3"
shift 3

FIO_ARGS=(
    "--name=${TITLE}"
    "--filename=$(echo "$@" | sed -n 's/. *file=\([^ ]*\).*\1/p')"
    "--rw=$( [ "$MODE" = "r" ] && echo "read" || echo "write" )"
    "--bs=${BS}"
    "--direct=1"
    "--ioengine=$(echo "$@" | sed -n 's/. *ioengine=\([^ ]*\).*\1/p')"
    "--iodepth=$(echo "$@" | sed -n 's/. *iodepth=\([^ ]*\).*\1/p')"
    "--numjobs=$(echo "$@" | sed -n 's/. *numjobs=\([^ ]*\).*\1/p')"
    "--group_reporting=1"
    "--time_based=0"

```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

"--output=logs/${TITLE}_${BS}_${MODE}_qd$(echo "$@" | sed -n 's/.*qd=\\([^\ ]*\\).*/\1/p').log"
)

# 透传额外参数（如 --readonly、--unlink=0 等）
for arg in "$@"; do
    case "$arg" in
        file=*|ioengine=*|iodepth=*|numjobs=*|qd=*)
            ;; # 已解析
        *)
            FIO_ARGS+=("$arg")
            ;;
    esac
done

mkdir -p logs
echo ">> Running fio with args:"
printf ' %s\n' "${FIO_ARGS[@]}"

fio "${FIO_ARGS[@]}"

// File: tools/clear_cache.sh
#!/usr/bin/env bash
# clear_cache.sh: 清空 Linux 页缓存 / inode / dentry 缓存

set -euo pipefail

echo 3 | sudo tee /proc/sys/vm/drop_caches >/dev/null
sync
echo "[cache] Dropped pagecache, dentries and inodes."

```

通过将“测试矩阵调度”与“底层 FIO 执行”拆分为 Python + Bash 两层，一方面便于在 Python 侧编写更复杂的测试组合与参数检查逻辑，另一方面又能保持底层 I/O 调用的透明和可控。clear_cache.sh 的存在则确保了 COLD READ 测试的可重复性。

5.2 性能测试结果与分析

5.2.1 QEMU 虚拟机性能测试

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

在 QEMU 虚拟机环境中，我们对比测试了“原生 F2FS 内核”和“引入 Large Folios + iomap 适配后的内核”在顺序读写大文件场景下的性能表现。测试文件大小远大于内存，以避免全部落在页缓存中；同时分别采集 COLD READ 与 WARM READ 情况下的带宽。所有其他参数（如磁盘类型、CPU 配置、内存大小等）保持一致。

从带宽结果可以看到，在顺序写测试中，Large Folios + iomap 方案明显提高了写入吞吐，这与第四章中“减少逐页处理开销、合并 I/O 提交”的设计目标是一致的；在顺序读测试中，COLD READ 下仍然有可观提升，说明新的 readahead 与 folio 生命周期管理并未引入额外瓶颈；WARM READ 场景下，改造后的内核也能更充分地利用缓存与大页带来的 TLB 优势。

【此处插入：图 5-1 QEMU 虚拟机环境下顺序读/写带宽对比（普通文件）】

【此处插入：图 5-2 QEMU 虚拟机环境下 COLD/WARM READ 带宽对比（普通文件）】

5.2.2 树莓派 5 性能测试

在树莓派 5 平台上，我们采用与虚拟机同样的脚本和方法，对基于 SD 卡 / eMMC 的 F2FS 分区进行顺序读写大文件测试。相比 QEMU，树莓派的平台更贴近 F2FS 的典型部署场景，同时其 CPU 性能与 I/O 带宽也更受限，更能体现算法改动对资源受限设备的影响。

实验结果表明，在树莓派上，改造后的 F2FS 同样在顺序写入场景中获得明显带宽提升，顺序读取尤其是 COLD READ 场景下也表现出更好的吞吐，这说明 Large Folios + iomap 方案不仅在桌面/虚拟机环境下有效，在嵌入式平台上依然能够带来稳定收益。同时，在长时间反复运行顺序写测试时，设备层写入量的增长曲线也更加平滑，侧面反映出写放大被一定程度上抑制。

【此处插入：图 5-3 树莓派 5 平台顺序读/写带宽对比（普通文件）】

【此处插入：图 5-4 树莓派 5 平台 COLD/WARM READ 带宽对比（普通文件）】

5.3 数据处理与可视化脚本

5.3.1 日志解析脚本：parse_fio_logs.py

所有 FIO 的执行结果都会输出到独立的 log 文件中。为了将这些日志统一整理成可分析的结构化数据，本项目实现了 parse_fio_logs.py 脚本，负责从 FIO 文本输出中解析出带宽、IOPS、延迟以及 CPU 利用率等关键指标，并汇总为 CSV 文件。

```
// File: tools/parse_fio_logs.py
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。


```
#!/usr/bin/env python3
# parse_fio_log.py: 将 FIO 文本日志解析为结构化 CSV 数据

import re
import csv
from pathlib import Path

LOG_DIR = Path("logs")
OUT_CSV = Path("results.csv")

def _parse_with_suffix(s: str) -> float:

    # 将带 K/M/G 后缀的数字统一换算为基础单位。
    # 例如 '10.5k' → 10500, '3M' → 3 * 1e6。

    s = s.strip()
    m = re.match(r"([0-9.]+\s*)([kKmMgG]?)", s)
    if not m:
        return float(s)
    val = float(m.group(1))
    suf = m.group(2).lower()
    if suf == "k":
        val *= 1e3
    elif suf == "m":
        val *= 1e6
    elif suf == "g":
        val *= 1e9
    return val

def parse_fio_log(path: Path, meta: dict):
    txt = path.read_text(encoding="utf-8", errors="ignore")

    # 1) 带宽 / IOPS / CPU 使用率
    bw_rg = re.search(r"bw=(\d+(?:\.\d+)?)\s*(MiB/s|GiB/s)", txt, flags=re.I)
    iops_rg = re.search(r"iops=([0-9.kKmMgG]+)", txt, flags=re.I)
    cpu_rg = re.search(r"cpu\s*=\s*usr\s*=\s*s*([0-9.]+)%\s*,\s*s*sys\s*=\s*s*([0-9.]+)%",
                        txt, flags=re.I)
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

# 2) 平均延迟 / 99% 延迟 (可选)
clat_rg = re.search(
    r"clat\s*\(((usec|msec)\):.*?avg=\s*([0-9.]+)",
    txt, flags=re.I | re.S
)
p99_rg = re.search(
    r"99.00th=\s*([0-9.]+)",
    txt, flags=re.I
)

# 校验必需字段
if not (bw_rg and iops_rg and cpu_rg):
    print(f"[warn] 必要字段缺失 → {path.name}")
    return None

# 单位换算
bw = float(bw_rg.group(1))
if "GiB/s" in bw_rg.group(2):
    bw *= 1024.0 # 统一成 MiB/s

iops = _parse_with_suffix(iops_rg.group(1))

clat_avg = None
clat_p99 = None
if clat_rg:
    clat_avg = float(clat_rg.group(2))
    if clat_rg.group(1).lower() == "msec":
        clat_avg *= 1000.0 # 统一成 usec

if p99_rg:
    clat_p99 = float(p99_rg.group(1))
    if "clat percentiles (msec)" in txt:
        clat_p99 *= 1000.0

cpu_usr = float(cpu_rg.group(1))
cpu_sys = float(cpu_rg.group(2))

return {
    **meta,
    "bw_mib_s": bw,

```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```

    "iops": iops,
    "clat_avg_us": clat_avg,
    "clat_p99_us": clat_p99,
    "cpu_usr_pct": cpu_usr,
    "cpu_sys_pct": cpu_sys,
}

def main():
    rows = []
    for p in LOG_DIR.glob("*.log"):
        # 从文件名中解析内核版本 / 块大小 / 模式 / qd 等元数据
        # 例如: vanilla_8G_mem_4k_r_qd1.log
        name = p.stem
        meta = {"raw_name": name}
        # 这里省略具体的正则拆分细节, 保持与决赛文档一致
        r = parse_fio_log(p, meta)
        if r:
            rows.append(r)

    if not rows:
        print("[warn] 未解析到任何有效行")
        return

    keys = sorted(rows[0].keys())
    with OUT_CSV.open("w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=keys)
        w.writeheader()
        w.writerows(rows)
    print(f"[ok] 写出 {len(rows)} 行到 {OUT_CSV}")

if __name__ == "__main__":
    main()

```

通过该脚本, 我们可以将大量 FIO 日志统一汇总为一份 CSV 文件, 后续便可直接调用 Python / Matplotlib 或其它工具进行绘图与统计分析。

5.3.2 结果可视化脚本: visualize_results.py

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点, 为第四章的解决方案设计提供了问题导向。

visualize_results.py 负责从 parse_fio_logs.py 生成的 CSV 中读取数据，按照内核版本、块大小、队列深度等维度绘制带宽对比图，生成类似决赛文档中“普通文件带宽对比图”的柱状图或折线图。脚本本身主要使用 pandas 做分组与聚合，使用 matplotlib 完成绘制和输出。

```
// File: tools/visualize_results.py

#!/usr/bin/env python3
# visualize_results.py: 从 CSV 绘制带宽对比图

import pandas as pd
import matplotlib.pyplot as plt

CSV_PATH = "results.csv"
OUT_DIR = "figs"

def plot_bw_by_kernel(df: pd.DataFrame, title_suffix: str, out_name: str):

    # 按内核版本分组画带宽柱状图。

    grouped = df.groupby(["kernel", "bs"])["bw_mib_s"].mean().reset_index()
    piv = grouped.pivot(index="bs", columns="kernel", values="bw_mib_s")

    ax = piv.plot(kind="bar")
    ax.set_xlabel("Block size")
    ax.set_ylabel("Bandwidth (MiB/s)")
    ax.set_title(f"Sequential throughput ({title_suffix})")
    ax.legend(title="Kernel")

    plt.tight_layout()
    plt.savefig(f"{OUT_DIR}/{out_name}.png")
    plt.close()

def main():
```

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。

```
df = pd.read_csv(CSV_PATH)
Path(OUT_DIR).mkdir(exist_ok=True)

# 示例：只对顺序读测试绘图
df_read = df[df["mode"] == "read"]
plot_bw_by_kernel(df_read, "read", "bw_read")

# 示例：顺序写
df_write = df[df["mode"] == "write"]
plot_bw_by_kernel(df_write, "write", "bw_write")

if __name__ == "__main__":
    main()
```

至此，从“自动化调度顺序读写测试脚本”到“日志解析与绘图脚本”，本章完整复现了决赛文档中的实验方法论和工具链。这些脚本不仅支撑了第五章中所有图表数据的获取，也为后续在更多平台上复现实验结果提供了基础。

¹ 本章详细剖析了在 F2FS 中引入 Large Folios 所面临的架构性冲突与实现难点，为第四章的解决方案设计提供了问题导向。